Microservice Design Patterns

A Complete Hands-On Deep-Dive Guide with Production-Grade Java / Spring Boot Code

Java 17 | Spring Boot 3.x | Spring Cloud | Apache Kafka | PostgreSQL | MongoDB | Cassandra | Docker Compose | Resilience4j

Table of Contents

1. How to Use This Guide & Environment Setup 3
2. Shared Infrastructure (Docker Compose: Kafka, Postgres, Mongo, Cassandra) 6
3. Pattern 1 — Database per Microservice 7
4. Pattern 2 — CQRS (Command Query Responsibility Segregation) 11
5. Pattern 3 — Event Sourcing 17
6. Pattern 4 — Saga Pattern (Choreography + Orchestration) 20
7. Pattern 5 — Backend for Frontend (BFF) 23
8. Pattern 6 — API Gateway 25
9. Pattern 7 — Strangler Fig Pattern 28
10. Pattern 8 — Circuit Breaker (Resilience4j) 30
11. Pattern 9 — Externalized Configuration (Spring Cloud Config) 32
12. Pattern 10 — Consumer-Driven Contract Testing (Pact) 34
13. Bonus Patterns — Chain Queries, Service Aggregator, Service Registry, Message Broker, Pub-Sub, Event-Driven Architecture & the Outbox Pattern 37
14. Pattern Selection Cheat Sheet 46
15. Common Pitfalls Across All Patterns 47

1. How to Use This Guide

This handbook is a **hands-on, code-first** expansion of the ten core microservice design patterns. Every pattern includes: a plain-English explanation of the problem it solves, a full runnable Spring Boot implementation (entities, repositories, services, controllers, Kafka producers/consumers where relevant), the exact `pom.xml` dependencies and `application.yml` configuration needed, a Docker Compose snippet for any required infrastructure, curl-based test commands, and a "gotchas" section covering the mistakes engineers make in production.

The tech stack used throughout is deliberately consistent so the examples compose into one coherent mental model of a real e-commerce platform with three bounded contexts: **Product** (polyglot persistence with MongoDB), **Order** (PostgreSQL, the transactional core), and **Customer** (PostgreSQL). Apache Kafka is the backbone for all asynchronous, event-driven communication. Cassandra is introduced specifically in the CQRS and Event Sourcing sections as the read-optimized / append-heavy store it is best suited for.

Concern	Technology	Why
Language / Runtime	Java 17, Spring Boot 3.2+	LTS, virtual-thread-ready, modern Spring Boot 3 baseline
Inter-service messaging	Apache Kafka 3.x	Durable log, replay-ability, natural fit for event sourcing/CQRS/sagas
Relational store	PostgreSQL 16	Strong consistency for Order/Customer aggregates, ACID transactions
Document store	MongoDB 7	Flexible schema for Product catalog, fast reads
Wide-column store	Apache Cassandra 4.x	Linear write scalability for CQRS read models and event stores
Resilience	Resilience4j	Circuit breaker, retry, bulkhead, rate limiter
Gateway	Spring Cloud Gateway	Reactive, non-blocking routing layer
Config	Spring Cloud Config Server	Externalized, Git-backed, refreshable configuration
Contract testing	Pact JVM	Consumer-driven contracts between Order and User/Customer services
Containerization	Docker & Docker Compose	Local multi-service orchestration mirroring production topology

> **Prerequisites:** JDK 17+, Maven 3.9+, Docker Desktop (or equivalent), at least 8GB RAM free for the Compose stack (Kafka + Zookeeper + Postgres + Mongo + Cassandra together are heavy — see Section 2 for a "lite" profile that swaps Cassandra/Kafka for lightweight test-containers when you just want to follow along on a laptop).

The Architecture Evolution Journey

Every pattern in this guide is best understood as one stop on a longer journey, not an isolated technique. Real systems typically evolve through five recognizable stages, each unlocked once the previous stage's capacity or maintainability ceiling is reached:

Stage	Shape	Typically outgrown when...
1. Monolithic	One codebase, one deployable, one shared database	the team can no longer deploy safely without a full regression pass, or a single DB becomes the bottleneck
2. Layered Monolith	Presentation / Business Logic / Data Access layers inside the same deployable	internal modules are cleanly separated in code but still fail and scale together
3. Service-Oriented (SOA)	Coarse-grained services, often sharing a database or an ESB	coordination overhead between a handful of large services becomes the new bottleneck
4. Microservices	Fine-grained, independently deployable services, each owning its own data (Pattern 1)	synchronous inter-service chatter (Bonus A) or fragile point-to-point integrations pile up as service count grows
5. Event-Driven Microservices	Services publish/consume domain events over a shared backbone (Bonus B & C), decoupling producers from consumers entirely	—the end state this guide builds toward

> **Don't skip stages prematurely.** A well-built monolith is still the right starting point for most new products — the patterns in this guide exist to solve problems that only appear once a system's scale, team size, or deployment cadence outgrows what one deployable and one database can comfortably handle. Introducing Sagas, CQRS, or an event backbone before that pain exists adds operational cost with no corresponding benefit (a violation of the YAGNI principle).

Non-Functional Requirements Driving Every Pattern Choice

Each pattern in this guide is ultimately in service of one or more of four non-functional requirements. Keeping these front-of-mind while reading makes it easier to judge whether a given pattern is worth its added complexity for your own system:

- **Scalability** — can the system absorb a growing number of concurrent users and requests without a proportional increase in latency or failure rate?
- **Availability** — does the system keep serving traffic through partial failures, deployments, and maintenance, ideally around the clock?
- **Maintainability** — can teams understand, change, and safely deploy their part of the system without deep knowledge of every other

part?

- **Efficiency (Latency)** — does the system respond within an acceptable time budget (a common target for user-facing APIs is well under 2 seconds, and often much less for individual service calls) even as load increases?

Capacity Planning — Matching Architecture to Expected Load

It's worth sizing the architecture to the load it actually needs to handle rather than over-engineering for hypothetical scale on day one. A useful way to reason about this is to map expected concurrent users and requests-per-second onto the architecture stage that comfortably supports them:

Concurrent Users	Approx. Requests/sec	Comfortable Architecture Stage	Acceptable Latency Target
~2,000	~500 req/s	Monolithic or Layered Monolith with a load balancer + horizontal scale-out	< 2s
~20,000	~5,000 req/s	Microservices with Database per Service (Pattern 1) + API Gateway (Pattern 6)	< 1s
~200,000	~50,000 req/s	Microservices + CQRS (Pattern 2) + Circuit Breaker (Pattern 8) + caching read paths	< 500ms
Millions	100,000+ req/s	Full Event-Driven Microservices with an event backbone (Bonus C), stream processing, and independently scaled read models	< 200ms per hop

> **How to use this table:** start each new system at the row matching your realistic near-term load (not your aspirational scale), and treat the pattern set in the next row down as your migration target once you're consistently pushing against the current stage's latency or availability ceiling — following the same Strangler Fig approach (Pattern 7) used for migrating legacy systems.

Where We're Coming From — The Monolithic Starting Point

Every one of these patterns is a response to the same starting point: a single deployable application backed by one shared database. Understanding that starting point makes each pattern's motivation concrete.

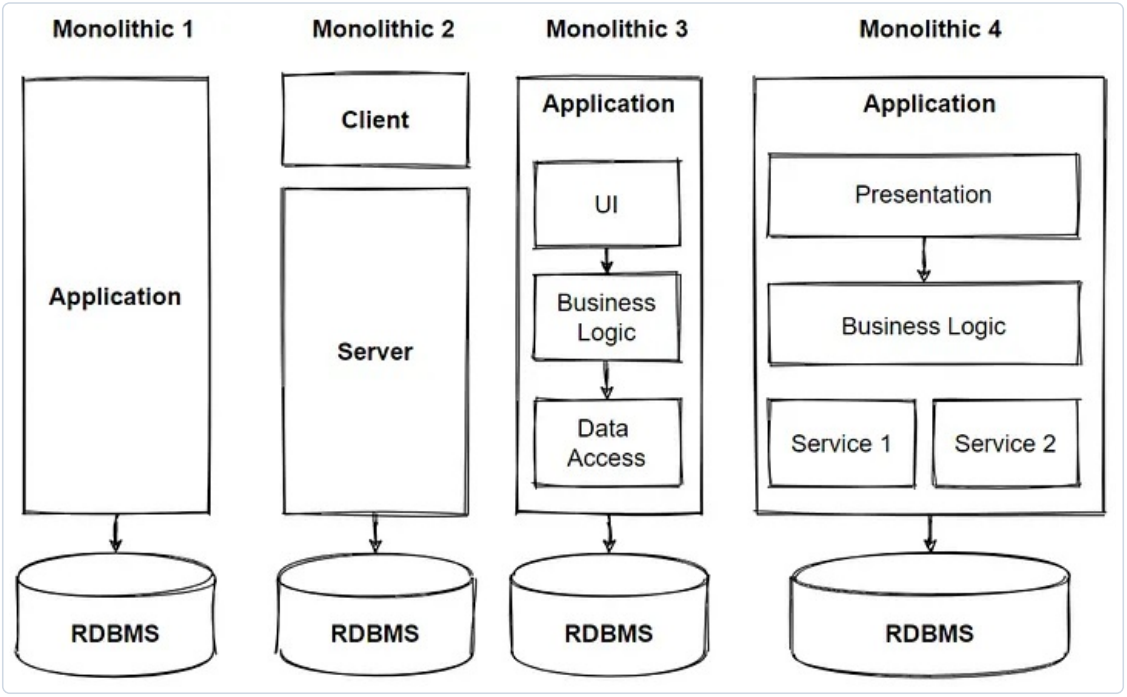


Fig. 1 — Four flavors of monolithic architecture, from a single deployable to a presentation/business-logic/service-oriented split — all still backed by one shared RDBMS.

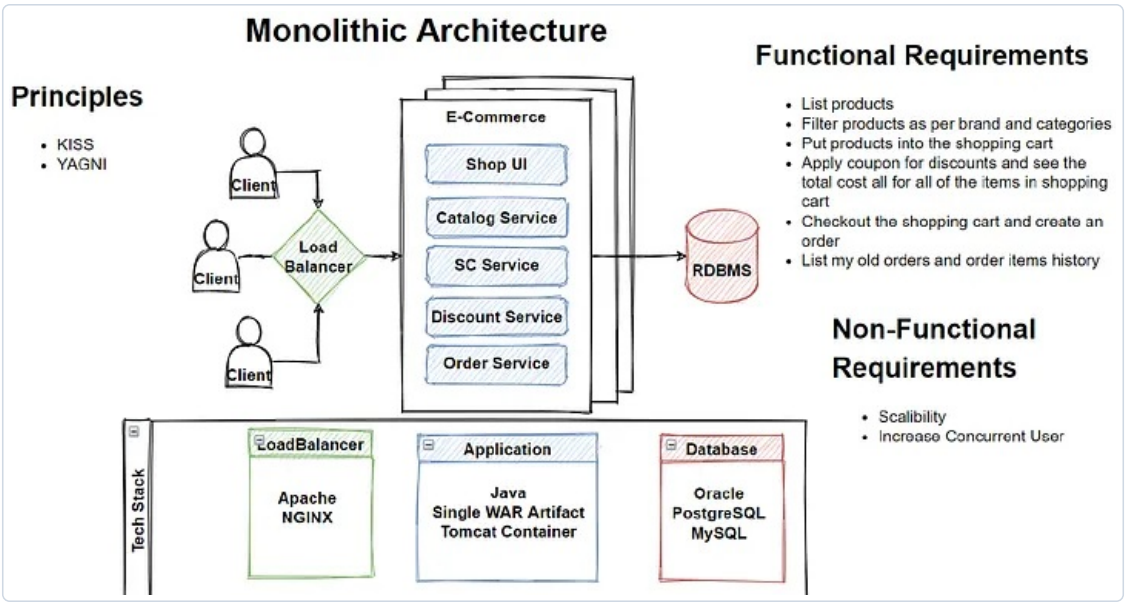


Fig. 2 — A concrete monolithic e-commerce application: one WAR artifact on Tomcat, one Oracle/PostgreSQL/MySQL database, serving Shop UI, Catalog, Shopping Cart, Discount, and Order logic all from a single codebase and schema — the exact structure microservices decompose in the patterns below.

2. Shared Infrastructure Setup

Every pattern below assumes the following Docker Compose stack is running. Save this as `docker-compose.yml` in your project root and run `docker compose up -d` once before starting any Spring Boot service.

docker-compose.yml

```
version: '3.8'
services:
  zookeeper:
    image: confluentinc/cp-zookeeper:7.6.0
    environment:
      ZOOKEEPER_CLIENT_PORT: 2181
    ports: ["2181:2181"]

  kafka:
    image: confluentinc/cp-kafka:7.6.0
    depends_on: [zookeeper]
    ports: ["9092:9092"]
    environment:
      KAFKA_BROKER_ID: 1
      KAFKA_ZOOKEEPER_CONNECT: zookeeper:2181
      KAFKA_ADVERTISED_LISTENERS: PLAINTEXT://kafka:29092,PLAINTEXT_HOST://localhost:9092
      KAFKA_LISTENER_SECURITY_PROTOCOL_MAP: PLAINTEXT:PLAINTEXT,PLAINTEXT_HOST:PLAINTEXT
      KAFKA_INTER_BROKER_LISTENER_NAME: PLAINTEXT
      KAFKA_OFFSETS_TOPIC_REPLICATION_FACTOR: 1

  postgres-order:
    image: postgres:16
    environment:
      POSTGRES_DB: orderdb
      POSTGRES_USER: order_user
      POSTGRES_PASSWORD: order_pass
    ports: ["5432:5432"]
    volumes: ["order_pg_data:/var/lib/postgresql/data"]

  postgres-customer:
    image: postgres:16
    environment:
      POSTGRES_DB: customerdb
      POSTGRES_USER: customer_user
      POSTGRES_PASSWORD: customer_pass
    ports: ["5433:5432"]

  mongo-product:
    image: mongo:7
    ports: ["27017:27017"]
    volumes: ["mongo_product_data:/data/db"]

  cassandra:
    image: cassandra:4.1
    ports: ["9042:9042"]
    environment:
      - CASSANDRA_CLUSTER_NAME=orders_cluster
    volumes: ["cassandra_data:/var/lib/cassandra"]

  config-server:
    build: ./config-server
    ports: ["8888:8888"]

volumes:
  order_pg_data:
  mongo_product_data:
  cassandra_data:
```

> **Lite profile:** If your laptop is resource-constrained, comment out the `cassandra` service while working through Patterns 1, 4, 5, 6, 7, 8, 9, 10 — Cassandra is only actively used as the CQRS/Event Sourcing read-store in Sections 4 and 5. Kafka + Zookeeper can also be replaced with Testcontainers' `KafkaContainer` for integration tests without running the full stack manually.

Initializing Cassandra keyspace (run once after the container is up)

```
docker exec -it <cassandra_container> cqlsh -e "
CREATE KEYSPACE IF NOT EXISTS orders_ks
WITH replication = {'class':'SimpleStrategy','replication_factor':1};
"
```

Parent Maven POM shared by all services

Each microservice is an independent Maven module with its own `pom.xml`, but they share common dependency versions via a parent POM to avoid version drift.

pom.xml (parent)

```
<project xmlns="http://maven.apache.org/POM/4.0.0">
  <modelVersion>4.0.0</modelVersion>
```

```
  <groupId>com.example</groupId>
  <artifactId>ecommerce-platform</artifactId>
  <version>1.0.0</version>
  <packaging>pom</packaging>

  <parent>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-parent</artifactId>
    <version>3.2.5</version>
  </parent>

  <properties>
    <java.version>17</java.version>
    <spring-cloud.version>2023.0.1</spring-cloud.version>
    <resilience4j.version>2.2.0</resilience4j.version>
  </properties>

  <modules>
    <module>product-service</module>
    <module>order-service</module>
    <module>customer-service</module>
    <module>bff-service</module>
    <module>api-gateway</module>
    <module>config-server</module>
  </modules>

  <dependencyManagement>
    <dependencies>
      <dependency>
        <groupId>org.springframework.cloud</groupId>
        <artifactId>spring-cloud-dependencies</artifactId>
        <version>${spring-cloud.version}</version>
        <type>pom</type>
        <scope>import</scope>
      </dependency>
    </dependencies>
  </dependencyManagement>
</project>
```

Pattern 1 — Database per Microservice Data Ownership

The Problem

In a monolith, every module reaches into the same shared schema. Changing an `orders` table column breaks the reporting module, the shipping module, and the invoicing module in one shot. Database per Microservice fixes this by giving each service exclusive ownership of its own schema/store — no other service is allowed to query it directly. All cross-service access happens through the owning service's API or through events.

Real-Time Usage in Production

- **E-commerce platforms (Amazon/Flipkart-style):** catalog service runs on a document store (MongoDB/Elasticsearch) for flexible, fast-changing product attributes, while the order/payment service runs on PostgreSQL/MySQL for strict ACID guarantees on money movement.
- **Ride-hailing apps (Uber/Ola-style):** the trip/location service uses a write-optimized store (Cassandra) for high-frequency GPS pings, while the billing/invoicing service uses a relational DB for transactional correctness.
- **Core banking systems:** the account/ledger service owns its Oracle/Postgres schema exclusively; the notification service owns a separate MongoDB store for templated messages, letting each team deploy and scale independently without a shared-schema change freeze.

Architecture

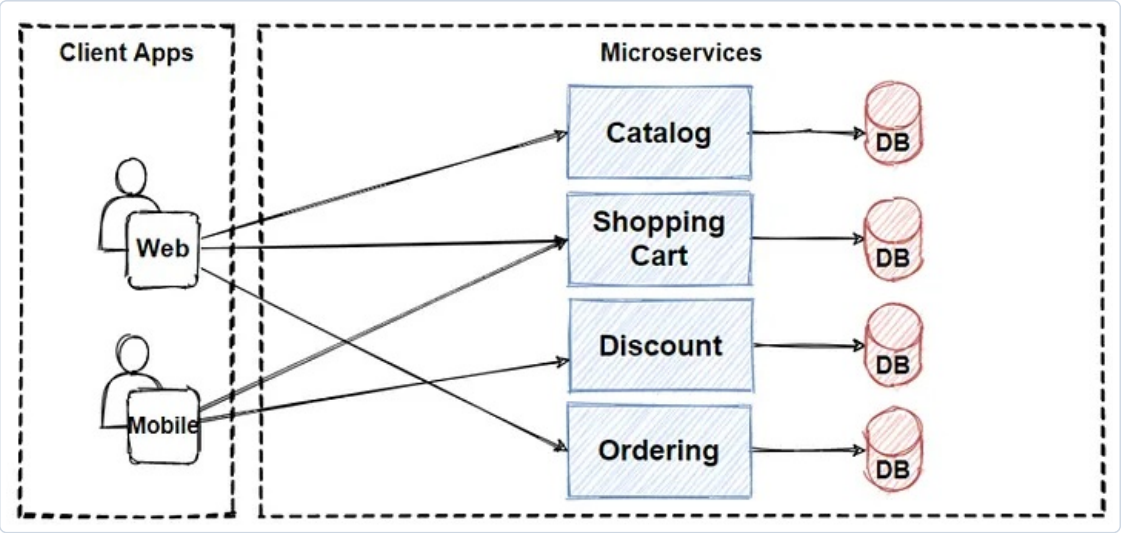


Fig. 3 — Each microservice (Catalog, Shopping Cart, Discount, Ordering) owns its own database exclusively; clients (Web, Mobile) call services directly and no service reaches into another's data store.

```
┌──────────────────┐        ┌──────────────────┐        ┌──────────────────┐
│  Product Service │        │  Order Service   │        │ Customer Service │
│  (Spring Boot)   │        │  (Spring Boot)   │        │  (Spring Boot)   │
└────────┬─────────┘        └────────┬─────────┘        └────────┬─────────┘
         │                           │                           │
         ▼                           ▼                           ▼
┌──────────────────┐        ┌──────────────────┐        ┌──────────────────┐
│    MongoDB       │        │   PostgreSQL     │        │   PostgreSQL     │
│  (product_db)    │        │   (orderdb)      │        │   (customerdb)   │
└──────────────────┘        └──────────────────┘        └──────────────────┘
  No service reaches into another service's database. Cross-cutting reads go through REST/Kafka.
```

Full Implementation — Product Service (MongoDB)

pom.xml (product-service)

```xml
<dependencies>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-data-mongodb</artifactId>
  </dependency>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-web</artifactId>
  </dependency>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-validation</artifactId>
  </dependency>
</dependencies>
```

src/main/resources/application.yml

```yaml
server:
  port: 8081
spring:
  application:
    name: product-service
  data:
    mongodb:
      uri: mongodb://localhost:27017/product_db
```

Product.java

```java
package com.example.product;

import org.springframework.data.annotation.Id;
import org.springframework.data.mongodb.core.index.Indexed;
import org.springframework.data.mongodb.core.mapping.Document;
import jakarta.validation.constraints.*;
import java.math.BigDecimal;

@Document(collection = "products")
public class Product {
    @Id
    private String id;

    @NotBlank
```

```
    private String name;

    @Indexed
    private String category;

    @NotNull @DecimalMin("0.01")
    private BigDecimal price;

    @Min(0)
    private int stockQuantity;

    // full getters/setters generated by Lombok @Data in real project
    public String getId() { return id; }
    public void setId(String id) { this.id = id; }
    public String getName() { return name; }
    public void setName(String name) { this.name = name; }
    public BigDecimal getPrice() { return price; }
    public void setPrice(BigDecimal price) { this.price = price; }
    public int getStockQuantity() { return stockQuantity; }
    public void setStockQuantity(int q) { this.stockQuantity = q; }
}
```

ProductRepository.java

```
package com.example.product;

import org.springframework.data.mongodb.repository.MongoRepository;
import java.util.List;

public interface ProductRepository extends MongoRepository<Product, String> {
    List<Product> findByCategory(String category);
    List<Product> findByStockQuantityLessThan(int threshold);
}
```

ProductService.java

```
@Service
public class ProductService {
    private final ProductRepository repository;

    public ProductService(ProductRepository repository) { this.repository = repository; }

    public Product create(Product product) {
        if (product.getStockQuantity() < 0)
            throw new IllegalArgumentException("Stock cannot be negative");
        return repository.save(product);
    }

    @Transactional(readOnly = true)
    public Product getById(String id) {
        return repository.findById(id)
            .orElseThrow(() -> new ProductNotFoundException(id));
    }

    public Product decrementStock(String id, int qty) {
        Product p = getById(id);
        if (p.getStockQuantity() < qty)
            throw new InsufficientStockException(id, qty, p.getStockQuantity());
        p.setStockQuantity(p.getStockQuantity() - qty);
        return repository.save(p);
    }
}
```

ProductController.java + GlobalExceptionHandler.java

```
@RestController
@RequestMapping("/products")
public class ProductController {
    private final ProductService service;
    public ProductController(ProductService service) { this.service = service; }

    @PostMapping
    @ResponseStatus(HttpStatus.CREATED)
    public Product create(@Valid @RequestBody Product product) {
        return service.create(product);
    }

    @GetMapping("/{id}")
    public Product get(@PathVariable String id) { return service.getById(id); }

    @PatchMapping("/{id}/stock/decrement")
    public Product decrement(@PathVariable String id, @RequestParam int qty) {
        return service.decrementStock(id, qty);
    }
}

@RestControllerAdvice
class GlobalExceptionHandler {
```

```
    @ExceptionHandler(ProductNotFoundException.class)
    @ResponseStatus(HttpStatus.NOT_FOUND)
    public Map<String, String> handleNotFound(ProductNotFoundException ex) {
        return Map.of("error", ex.getMessage());
    }

    @ExceptionHandler(InsufficientStockException.class)
    @ResponseStatus(HttpStatus.CONFLICT)
    public Map<String, String> handleConflict(InsufficientStockException ex) {
        return Map.of("error", ex.getMessage());
    }
}
```

Order Service Counterpart (PostgreSQL, JPA)

The Order service must never open a JDBC connection to `product_db`. Instead it calls Product Service's REST API (or reacts to product events) — this is the enforcement mechanism that keeps the databases truly decoupled.

application.yml (order-service)

```
server:
  port: 8082
spring:
  application:
    name: order-service
  datasource:
    url: jdbc:postgresql://localhost:5432/orderdb
    username: order_user
    password: order_pass
  jpa:
    hibernate:
      ddl-auto: validate
    properties:
      hibernate.dialect: org.hibernate.dialect.PostgreSQLDialect
  flyway:
    enabled: true
```

V1__init_orders.sql (Flyway migration)

```
CREATE TABLE orders (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    customer_id UUID NOT NULL,
    status VARCHAR(20) NOT NULL DEFAULT 'PENDING',
    total_amount NUMERIC(12,2) NOT NULL,
    created_at TIMESTAMP NOT NULL DEFAULT now()
);
CREATE TABLE order_items (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    order_id UUID NOT NULL REFERENCES orders(id) ON DELETE CASCADE,
    product_id VARCHAR(64) NOT NULL,
    quantity INT NOT NULL CHECK (quantity > 0),
    unit_price NUMERIC(12,2) NOT NULL
);
```

Order.java (JPA entity)

```
@Entity @Table(name = "orders")
public class Order {
    @Id @GeneratedValue
    private UUID id;
    private UUID customerId;
    @Enumerated(EnumType.STRING)
    private OrderStatus status = OrderStatus.PENDING;
    private BigDecimal totalAmount;
    @OneToMany(mappedBy = "order", cascade = CascadeType.ALL, orphanRemoval = true)
    private List<OrderItem> items = new ArrayList<>();
    // getters/setters omitted for brevity
}
```

Calling Product Service from Order Service — the correct way

```
@Service
public class ProductClient {
    private final RestClient restClient;

    public ProductClient(@Value("${services.product.url}") String baseUrl) {
        this.restClient = RestClient.builder().baseUrl(baseUrl).build();
    }

    public ProductDto fetchProduct(String productId) {
        return restClient.get()
            .uri("/products/{id}", productId)
            .retrieve()
```

```
            .body(ProductDto.class);
    }
}
```

Testing It End to End

```
# start infra
docker compose up -d mongo-product postgres-order

# create a product
curl -X POST http://localhost:8081/products \
  -H "Content-Type: application/json" \
  -d '{"name":"Wireless Mouse","category":"Electronics","price":19.99,"stockQuantity":50}'

# decrement stock (simulating an order line)
curl -X PATCH "http://localhost:8081/products/{id}/stock/decrement?qty=2"

# create an order referencing that product id via order-service
curl -X POST http://localhost:8082/orders \
  -H "Content-Type: application/json" \
  -d '{"customerId":"c1","items":[{"productId":"{id}","quantity":2,"unitPrice":19.99}]}'
```

Common pitfalls:

- Sharing a "common" library that contains JPA `@Entity` classes across services — this silently recreates a shared schema through the back door.
- Using synchronous REST calls for every cross-service read creates tight temporal coupling; prefer caching product data locally in Order service via Kafka events (see CQRS & Event Sourcing sections) for high-traffic read paths.
- Forgetting to version your migrations (Flyway/Liquibase) per-service — each service's schema evolves independently and must have its own migration history table.

When to use: Almost always for greenfield microservices. The only real trade-off is the loss of cross-database joins and distributed ACID transactions — which is exactly why patterns like Saga, CQRS, and Event Sourcing exist later in this guide.

Pattern 2 — CQRS (Command Query Responsibility Segregation) Read/Write Split

The Problem

A single Order model has to satisfy both a write-heavy checkout flow (strict validation, ACID) and a read-heavy "my orders" dashboard (denormalized, must scale horizontally, eventually consistent is fine). Forcing both through one schema means every index you add to speed up reads slows down writes, and vice versa. CQRS splits the model in two: a **write model** (PostgreSQL, normalized, transactional) and a **read model** (Cassandra, denormalized, optimized purely for query patterns), kept in sync via Kafka events.

This tension is easiest to see in a monolithic, single-database setup: that one database has to serve both complex, multi-table join queries for reporting and dashboards, *and* fast, validated CRUD operations for everyday writes. A query joining across ten tables can lock rows long enough to visibly slow down concurrent writes; a write path with heavy validation and business logic can hold locks long enough to slow down concurrent reads. As either side of that workload grows, the single shared schema becomes the bottleneck for both. CQRS's answer is to stop trying to serve both needs from one schema at all — commands (task-based writes like "add item to cart" or "checkout order") go through a schema optimized for validated, transactional updates, while queries are served from an entirely separate, denormalized schema optimized purely for the read patterns your UI actually needs, with no joins required at read time.

Naming note — CQRS's read side is also called the Materialized View pattern. The denormalized read database in a CQRS setup is frequently referred to on its own as a **Materialized View**: a pre-computed, query-ready projection of the write side's data, kept up to date asynchronously rather than computed fresh on every read.

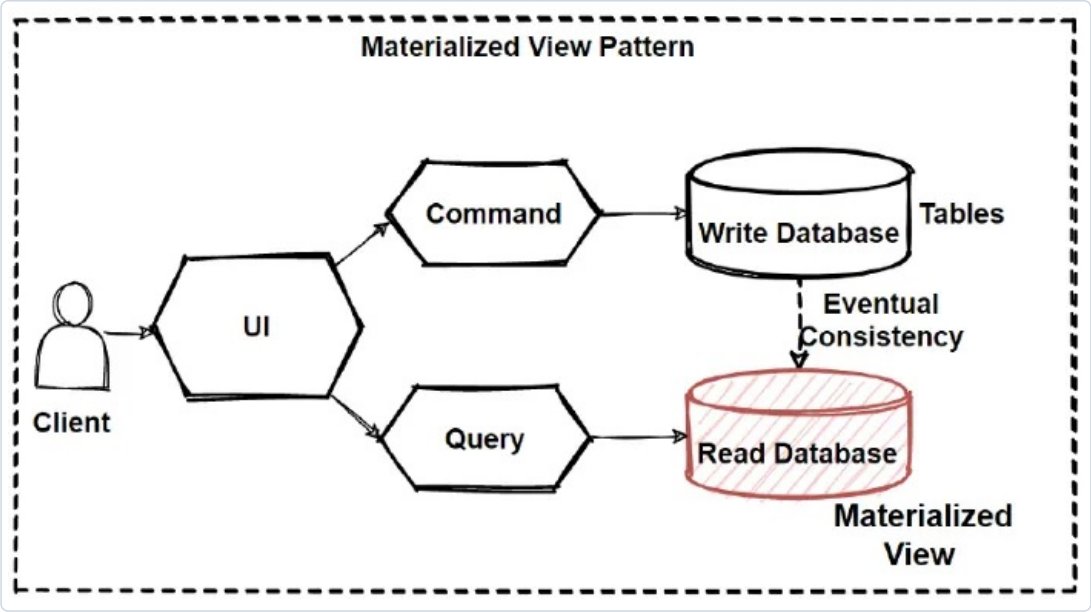


Fig. 3a — The same Command/Query split, viewed through the Materialized View lens: writes go through Command into Write Database Tables; the Read Database is explicitly labeled as the Materialized View that Query reads from, kept eventually consistent with the write side.

Real-Time Usage in Production

- **E-commerce order dashboards:** checkout writes go to a normalized PostgreSQL write model, while "my orders" / order-history search is served from a denormalized Elasticsearch or Cassandra read model that scales independently of checkout traffic.
- **Banking transaction/statement systems:** the core ledger accepts writes under strict ACID rules, while monthly statements and spend-analytics dashboards are generated from asynchronously updated read replicas or materialized views.
- **Social media feeds:** a post/like/comment write goes to the primary store, but the personalized home feed a user scrolls through is served from a separately optimized, pre-computed read store (classic CQRS at Twitter/Instagram feed-generation scale).

Real-World Case Study: Instagram's Database Architecture

Instagram's own backend is a widely cited real-world example of exactly this read/write split. At the core sits a Django application talking to several different data stores, each chosen for the workload it serves best — this is CQRS (and polyglot persistence, Pattern 1) at large scale, in production.

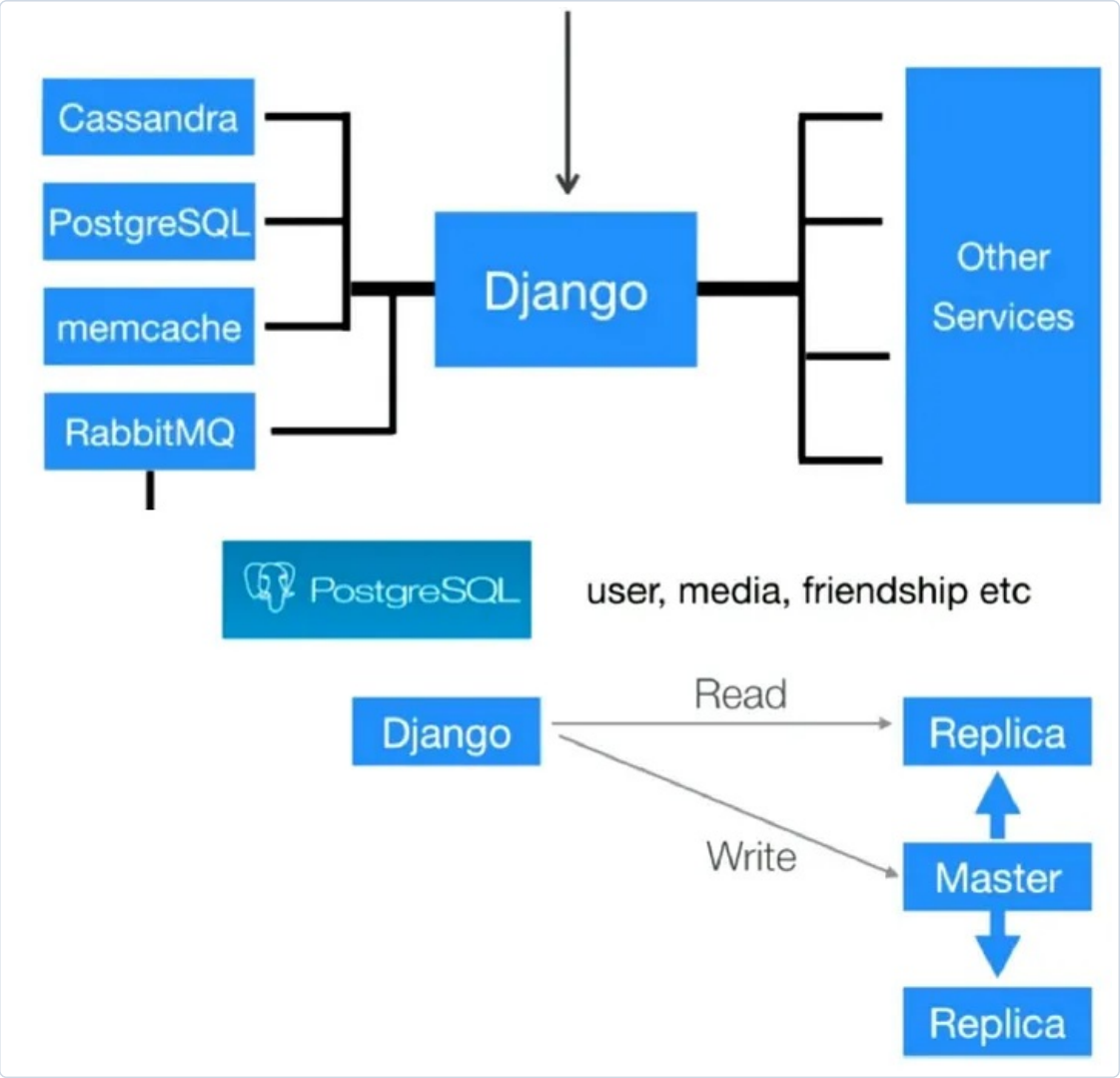


Fig. 3b — Django sits between multiple specialized stores — Cassandra, PostgreSQL, memcache, and RabbitMQ — on the read/write side, and fans out to Other Services on the other. PostgreSQL itself (holding user, media, and friendship data) is further split so Django reads from a Replica while all writes go to the Master, which then propagates to its replicas.

- **Cassandra** handles read-intensive, high-volume data like user stories/activity feeds — exactly the shape of workload a wide-column store is built for.
- **PostgreSQL** holds core relational data — user accounts, media metadata, friendship/follow relationships — where strong consistency and relational integrity matter more than raw read throughput.
- **memcache** sits in front of the hot-path reads as a caching layer, absorbing repeat queries before they ever reach a database.
- **RabbitMQ** handles the asynchronous messaging between components, playing the same role Kafka plays in this guide's own architecture.

The PostgreSQL read/write split shown at the bottom of Fig. 3b is worth calling out on its own: this is a simpler, lighter-weight alternative to a full CQRS implementation with two entirely different database technologies. Instead of standing up a separate NoSQL read store, Django simply writes to a **Master** node and reads from a **Replica** — the same underlying relational schema, replicated for read scaling, with no event-driven sync pipeline required. This read-replica approach is frequently the right starting point before committing to a full heterogeneous CQRS setup: it captures most of the "scale reads independently of writes" benefit with a fraction of the operational complexity, and is a natural first step before migrating to a genuinely denormalized, purpose-built read model once query patterns are well understood.

> **Choosing between the two approaches:** reach for a simple read replica first when your read queries still look like your write schema (you just need more read capacity). Reach for full CQRS with a separate, denormalized store once your read patterns have diverged enough from your write schema that no amount of replication or indexing on the write-side schema would make those queries fast.

Architecture

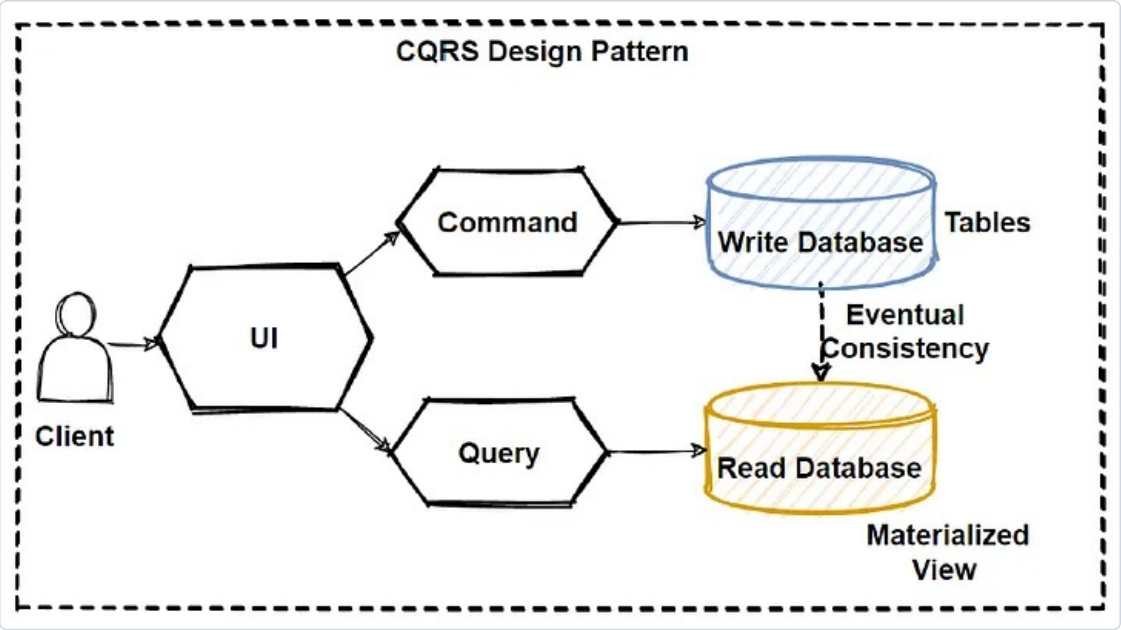


Fig. 4 — CQRS at the request level: the UI splits into a Command path (writes to the Write Database / tables) and a Query path (reads from a separate Read Database / materialized view), linked by eventual consistency.

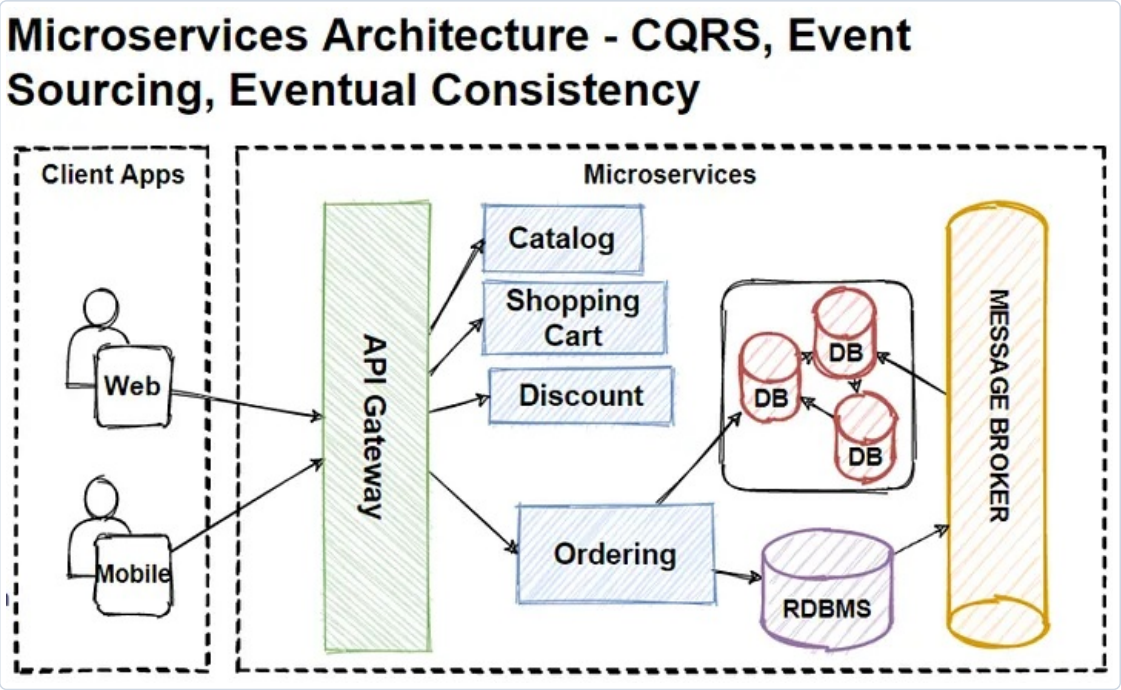


Fig. 5 — CQRS combined with Event Sourcing at the platform level: Ordering's write side persists to its own databases and publishes to a message broker, which other services and read models consume asynchronously.

```
  Client
    |  POST /orders (command)          |  GET /orders/{id} (query)
    ▼                                  ▼
┌──────────────┐                  ┌──────────────────────┐
| Command side |                  |  Query side          |
| OrderCommand-|                  |  OrderQuery-         |
| Controller   |                  |  Controller          |
└──────┬───────┘                  └──────────┬───────────┘
       | writes                              | reads
       ▼                                     ▼
┌──────────────┐   Kafka topic    ┌──────────────────────┐
| PostgreSQL   |──"order-events"─→|  Cassandra           |
| (orderdb)    |  OrderEventListener|  (orders_read_ks)  |
└──────────────┘   consumes & projects└──────────────────┘
```

Write Side — Command Model (PostgreSQL)

OrderCommandController.java

```java
@RestController
@RequestMapping("/orders")
public class OrderCommandController {
    private final OrderCommandService commandService;

    public OrderCommandController(OrderCommandService s) { this.commandService = s; }
```

```
    @PostMapping
    @ResponseStatus(HttpStatus.CREATED)
    public OrderResponse create(@Valid @RequestBody CreateOrderCommand cmd) {
        return commandService.handle(cmd);
    }

    @PostMapping("/{id}/cancel")
    public void cancel(@PathVariable UUID id) { commandService.cancel(id); }
}
```

OrderCommandService.java — publishes structured events, not strings

```
@Service
public class OrderCommandService {
    private final OrderRepository orderRepository;
    private final KafkaTemplate<String, OrderEvent> kafkaTemplate;
    private static final String TOPIC = "order-events";

    public OrderCommandService(OrderRepository r, KafkaTemplate<String, OrderEvent> k) {
        this.orderRepository = r; this.kafkaTemplate = k;
    }

    @Transactional
    public OrderResponse handle(CreateOrderCommand cmd) {
        Order order = Order.from(cmd);
        order = orderRepository.save(order);

        OrderEvent event = new OrderEvent(
            order.getId(), "ORDER_CREATED", order.getCustomerId(),
            order.getTotalAmount(), order.getStatus().name(), Instant.now());

        // key by orderId so all events for one order land on the same partition (ordering guarantee)
        kafkaTemplate.send(TOPIC, order.getId().toString(), event);
        return OrderResponse.from(order);
    }

    @Transactional
    public void cancel(UUID id) {
        Order order = orderRepository.findById(id).orElseThrow(() -> new OrderNotFoundException(id));
        order.setStatus(OrderStatus.CANCELLED);
        orderRepository.save(order);
        kafkaTemplate.send(TOPIC, id.toString(),
            new OrderEvent(id, "ORDER_CANCELLED", order.getCustomerId(),
                           order.getTotalAmount(), "CANCELLED", Instant.now()));
    }
}
```

OrderEvent.java (shared Kafka payload, JSON via Spring Kafka's JsonSerializer)

```
public record OrderEvent(
    UUID orderId, String type, UUID customerId,
    BigDecimal totalAmount, String status, Instant occurredAt
) {}
```

Read Side — Query Model (Cassandra)

Cassandra models are designed *around the query*, not around normalized entities. Here the dominant query is "list a customer's orders sorted by newest first," so `customer_id` is the partition key and `created_at` the clustering column.

pom.xml additions for Cassandra + Kafka

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-data-cassandra</artifactId>
</dependency>
<dependency>
  <groupId>org.springframework.kafka</groupId>
  <artifactId>spring-kafka</artifactId>
</dependency>
```

schema.cql

```
CREATE TABLE IF NOT EXISTS orders_ks.order_by_customer (
    customer_id UUID,
    created_at TIMESTAMP,
    order_id UUID,
    status TEXT,
    total_amount DECIMAL,
    PRIMARY KEY ((customer_id), created_at)
) WITH CLUSTERING ORDER BY (created_at DESC);

CREATE TABLE IF NOT EXISTS orders_ks.order_by_id (
    order_id UUID PRIMARY KEY,
    customer_id UUID,
```

```
    status TEXT,
    total_amount DECIMAL,
    created_at TIMESTAMP
);
```

application.yml (query-side)

```
spring:
  cassandra:
    keyspace-name: orders_ks
    contact-points: localhost
    port: 9042
    local-datacenter: datacenter1
  kafka:
    bootstrap-servers: localhost:9092
    consumer:
      group-id: order-read-model-projector
      value-deserializer: org.springframework.kafka.support.serializer.JsonDeserializer
      properties:
        spring.json.trusted.packages: com.example.order.events
```

OrderByCustomer.java (Cassandra entity)

```
@Table("order_by_customer")
public class OrderByCustomer {
    @PrimaryKeyColumn(name = "customer_id", type = PrimaryKeyType.PARTITIONED)
    private UUID customerId;
    @PrimaryKeyColumn(name = "created_at", ordering = Ordering.DESCENDING)
    private Instant createdAt;
    private UUID orderId;
    private String status;
    private BigDecimal totalAmount;
    // getters/setters
}
```

OrderEventListener.java — the projector

```
@Component
public class OrderEventListener {
    private final OrderByCustomerRepository byCustomerRepo;
    private final OrderByIdRepository byIdRepo;

    public OrderEventListener(OrderByCustomerRepository c, OrderByIdRepository i) {
        this.byCustomerRepo = c; this.byIdRepo = i;
    }

    @KafkaListener(topics = "order-events", groupId = "order-read-model-projector")
    public void onOrderEvent(OrderEvent event) {
        switch (event.type()) {
            case "ORDER_CREATED" -> project(event);
            case "ORDER_CANCELLED" -> updateStatus(event);
            default -> { /* ignore unknown event types, log for visibility */ }
        }
    }

    private void project(OrderEvent e) {
        var row = new OrderByCustomer();
        row.setCustomerId(e.customerId());
        row.setCreatedAt(e.occurredAt());
        row.setOrderId(e.orderId());
        row.setStatus(e.status());
        row.setTotalAmount(e.totalAmount());
        byCustomerRepo.save(row);
        byIdRepo.save(OrderById.from(e));
    }

    private void updateStatus(OrderEvent e) {
        // Cassandra has no update-in-place-by-secondary-lookup, so we re-write the row(s)
        byIdRepo.findById(e.orderId()).ifPresent(existing -> {
            existing.setStatus(e.status());
            byIdRepo.save(existing);
        });
    }
}
```

OrderQueryController.java

```
@RestController
@RequestMapping("/orders/query")
public class OrderQueryController {
    private final OrderByCustomerRepository byCustomerRepo;
    private final OrderByIdRepository byIdRepo;

    @GetMapping("/customer/{customerId}")
    public List<OrderByCustomer> forCustomer(@PathVariable UUID customerId) {
        return byCustomerRepo.findByCustomerId(customerId);
```

```
    }

    @GetMapping("/{orderId}")
    public OrderById one(@PathVariable UUID orderId) {
        return byIdRepo.findById(orderId).orElseThrow(() -> new OrderNotFoundException(orderId));
    }
}
```

Testing the Eventual Consistency Window

```
# 1. issue the command
curl -X POST http://localhost:8082/orders -H "Content-Type: application/json" \
  -d '{"customerId":"11111111-1111-1111-1111-111111111111","items":[...]}'

# 2. immediately query — you may (rarely) get a 404/empty list here, that is EXPECTED
curl http://localhost:8083/orders/query/customer/11111111-1111-1111-1111-111111111111

# 3. after ~10-100ms (Kafka consumer lag), the read model is populated
```

Gotchas:

- **Eventual consistency is a UX decision, not just a technical one.** If the user expects to see their order immediately after checkout, either read-your-writes from the command side for that one call, or use optimistic UI updates on the frontend.
- Cassandra has no joins and no secondary-index-friendly ad-hoc queries — every new query pattern usually means a new denormalized table (as shown with `order_by_customer` vs `order_by_id`).
- Always key Kafka messages by aggregate id (`orderId`) — without a partition key, events for the same order can be processed out of order by different consumer threads, corrupting the projection.

When to use: Read:write ratio is heavily skewed, read patterns are very different from write patterns (e.g., dashboards, search, reporting), or the write side has heavy domain logic that shouldn't be tangled with query optimization concerns.

Pattern 3 — Event Sourcing Append-Only Truth

The Problem

A normal CRUD table only stores *current state* — once you update a row, the previous value is gone. For domains where the history matters as much as the present (financial ledgers, order lifecycle, audit-heavy systems), Event Sourcing stores every state transition as an immutable event. Current state is never stored directly; it is *derived* by replaying events. This gives you a perfect audit trail, free time-travel debugging, and the ability to rebuild new read models from scratch at any time.

Real-Time Usage in Production

- **Banking / fintech ledgers:** every debit and credit is stored as an immutable event, never overwritten — required for regulatory audit trails and to reconstruct an account's balance at any point in history.
- **E-commerce order lifecycle:** replaying an order's events (created, item added, payment confirmed, shipped, returned) lets customer support see exactly what happened and when, without a separate audit-log table to keep in sync.
- **Inventory / supply-chain tracking:** every stock movement (received, reserved, shipped, returned) is stored as an event, giving full traceability for compliance and enabling recalculation of stock levels if a reconciliation bug is found later.

Architecture

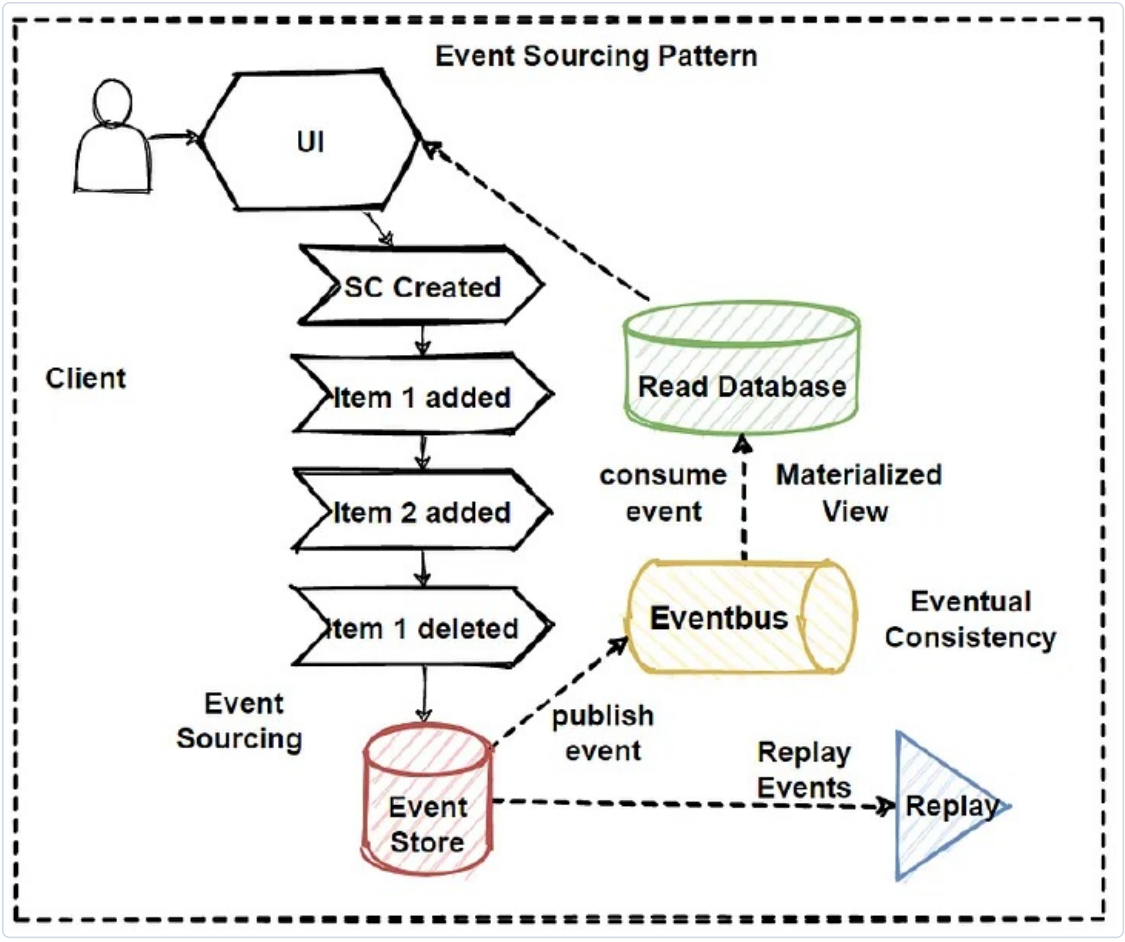


Fig. 6 — Every UI action (SC Created, Item added, Item deleted) is appended to the Event Store rather than overwriting state; events are published to an Eventbus and consumed to build a Materialized View in the Read Database, which the UI queries back — with Replay available to rebuild state from the full event history at any time.

```
Command ──► OrderAggregate (in memory) ──► validates ──► emits Event(s)
                                                              │
                                                              ▼
                                          ┌──────────────────────────────────┐
                                          │  Event Store (Cassandra)         │
                                          │  append-only, partitioned by     │
                                          │  aggregate_id, ordered by        │
                                          │  sequence_number                 │
                                          └──────────────────┬───────────────┘
                                                             │ replay
                                                             ▼
                                          Current state = fold(events, emptyOrder)
```

Why Cassandra for the Event Store

An event store is a pure append workload with a single dominant access pattern: "give me all events for aggregate X in order." That maps perfectly onto Cassandra's partition (aggregate id) + clustering (sequence number) model, and Cassandra's write path is optimized for exactly this kind of sequential, high-throughput append.

schema.cql (event store)

```
CREATE TABLE IF NOT EXISTS orders_ks.event_store (
    aggregate_id UUID,
    sequence_number BIGINT,
    event_type TEXT,
    payload TEXT,           -- JSON-serialized event data
    occurred_at TIMESTAMP,
    PRIMARY KEY ((aggregate_id), sequence_number)
) WITH CLUSTERING ORDER BY (sequence_number ASC);
```

pom.xml additions

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-data-cassandra</artifactId>
</dependency>
<dependency>
  <groupId>com.fasterxml.jackson.core</groupId>
  <artifactId>jackson-databind</artifactId>
</dependency>
```

Domain Events

DomainEvent.java / OrderCreatedEvent.java / ItemAddedEvent.java / OrderCancelledEvent.java

```
public sealed interface DomainEvent
    permits OrderCreatedEvent, ItemAddedEvent, OrderCancelledEvent {
    UUID aggregateId();
    Instant occurredAt();
}

public record OrderCreatedEvent(
    UUID aggregateId, UUID customerId, Instant occurredAt) implements DomainEvent {}

public record ItemAddedEvent(
    UUID aggregateId, String productId, int quantity,
    BigDecimal unitPrice, Instant occurredAt) implements DomainEvent {}

public record OrderCancelledEvent(
    UUID aggregateId, String reason, Instant occurredAt) implements DomainEvent {}
```

The Event Store (Cassandra-backed, generic)

```
@Repository
public class CassandraEventStore {
    private final CqlSession session;
    private final ObjectMapper mapper;
    private final Map<String, Class<? extends DomainEvent>> typeRegistry = Map.of(
        "OrderCreatedEvent", OrderCreatedEvent.class,
        "ItemAddedEvent", ItemAddedEvent.class,
        "OrderCancelledEvent", OrderCancelledEvent.class
    );

    public CassandraEventStore(CqlSession session, ObjectMapper mapper) {
        this.session = session; this.mapper = mapper;
    }

    public void append(UUID aggregateId, long expectedVersion, DomainEvent event) {
        // optimistic concurrency: only insert if this sequence number doesn't already exist
        ResultSet rs = session.execute(SimpleStatement.newInstance(
            "INSERT INTO orders_ks.event_store (aggregate_id, sequence_number, event_type, payload, occurred_at) " +
            "VALUES (?, ?, ?, ?, ?) IF NOT EXISTS",
            aggregateId, expectedVersion + 1, event.getClass().getSimpleName(),
            toJson(event), event.occurredAt()));
        if (!rs.wasApplied())
            throw new ConcurrentModificationException(
                "Aggregate " + aggregateId + " was modified concurrently — retry");
    }

    public List<DomainEvent> loadEvents(UUID aggregateId) {
        ResultSet rs = session.execute(SimpleStatement.newInstance(
            "SELECT event_type, payload FROM orders_ks.event_store WHERE aggregate_id = ? ORDER BY sequence_number ASC",
            aggregateId));
        List<DomainEvent> events = new ArrayList<>();
        for (Row row : rs) {
            Class<? extends DomainEvent> type = typeRegistry.get(row.getString("event_type"));
            events.add(fromJson(row.getString("payload"), type));
        }
        return events;
    }

    private String toJson(DomainEvent e) { try { return mapper.writeValueAsString(e); }
        catch (Exception ex) { throw new RuntimeException(ex); } }
    private DomainEvent fromJson(String json, Class<? extends DomainEvent> type) {
        try { return mapper.readValue(json, type); }
        catch (Exception ex) { throw new RuntimeException(ex); }
    }
}
```

The Aggregate — replay logic lives here

```
public class OrderAggregate {
    private UUID id;
    private UUID customerId;
    private final List<OrderLine> lines = new ArrayList<>();
    private String status = "OPEN";
    private long version = 0;

    public static OrderAggregate replay(List<DomainEvent> events) {
        OrderAggregate agg = new OrderAggregate();
        for (DomainEvent e : events) { agg.apply(e); agg.version++; }
        return agg;
    }

    private void apply(DomainEvent e) {
        switch (e) {
            case OrderCreatedEvent oc -> { this.id = oc.aggregateId(); this.customerId = oc.customerId(); }
```

```
            case ItemAddedEvent ia -> lines.add(new OrderLine(ia.productId(), ia.quantity(), ia.unitPrice()));
            case OrderCancelledEvent oca -> this.status = "CANCELLED";
        }
    }

    // command handlers validate against CURRENT (replayed) state before returning new events
    public ItemAddedEvent addItem(String productId, int qty, BigDecimal price) {
        if ("CANCELLED".equals(status))
            throw new IllegalStateException("Cannot add items to a cancelled order");
        return new ItemAddedEvent(id, productId, qty, price, Instant.now());
    }

    public long getVersion() { return version; }
}
```

Application Service — ties command → aggregate → store

```
@Service
public class OrderEventSourcedService {
    private final CassandraEventStore store;

    public UUID createOrder(UUID customerId) {
        UUID id = UUID.randomUUID();
        store.append(id, 0, new OrderCreatedEvent(id, customerId, Instant.now()));
        return id;
    }

    public void addItem(UUID orderId, String productId, int qty, BigDecimal price) {
        List<DomainEvent> history = store.loadEvents(orderId);
        OrderAggregate agg = OrderAggregate.replay(history);
        ItemAddedEvent newEvent = agg.addItem(productId, qty, price);
        store.append(orderId, agg.getVersion(), newEvent); // throws on concurrent write
    }

    public OrderAggregate currentState(UUID orderId) {
        return OrderAggregate.replay(store.loadEvents(orderId));
    }
}
```

Snapshots — avoiding "replay the last 50,000 events" pain

Once an aggregate accumulates thousands of events, replaying from scratch on every read gets expensive. The standard fix is periodic snapshotting: persist the folded state every N events, and on load, start from the latest snapshot instead of event zero.

```
@Table("order_snapshots")
public class OrderSnapshot {
    @PrimaryKey private UUID aggregateId;
    private long version;
    private String stateJson;
}

// load = snapshot (if any) + events with sequence_number > snapshot.version, then replay only the delta
```

Rebuilding a Read Model from History (killer feature)

```
# replay every event ever recorded to build a brand-new Cassandra/Postgres read table,
# e.g. after discovering a bug in the original projector logic:
events = eventStore.loadAllEventsOrderedByTime()
for event in events:
    newProjector.apply(event)
# this is impossible in a plain CRUD system — the history simply doesn't exist there
```

Gotchas:

- Event schemas are forever. Once `ItemAddedEvent` ships to production, you can only add fields (with defaults) — never rename or remove, or old events become unreadable. Use a version field and upcasters for real schema evolution.
- Replaying without snapshots on a hot aggregate (e.g., a long-lived shopping cart) becomes an O(n) tax on every single read — always add snapshotting once event counts pass a few hundred per aggregate.
- The optimistic-concurrency `IF NOT EXISTS` pattern above is a lightweight transaction in Cassandra terms — it uses Paxos internally and is noticeably slower than a plain insert; only rely on it at the aggregate root, not on every child entity.

When to use: Regulatory/audit-heavy domains, anything where "how did we get to this state" matters (fraud detection, financial ledgers), or when you want CQRS read models that can be rebuilt/replayed at will. Skip it for simple CRUD-shaped domains — the operational complexity is real.

Pattern 4 — Saga Pattern Distributed Transactions

The Problem

Checkout touches Order, Customer (credit/payment), and Product (stock) services — each with its own database (Pattern 1). A distributed two-phase-commit transaction across all three is fragile and doesn't scale. The Saga pattern instead breaks the transaction into a sequence of local transactions, each with a **compensating action** that undoes it if a later step fails.

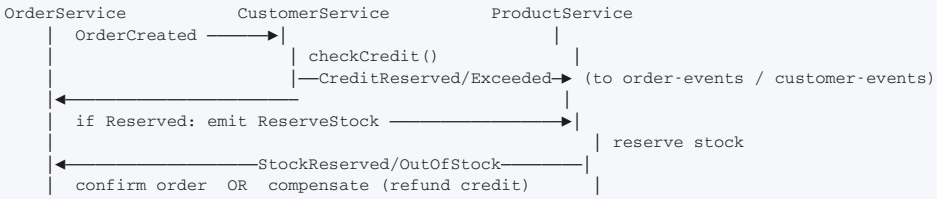
Two Flavors

Style	How it works	Trade-off
Choreography	Each service listens for the previous service's event and reacts, publishing its own event in turn. No central coordinator.	Simple to start, but the overall flow is implicit — hard to see "the saga" in one place as steps grow.
Orchestration	A dedicated Saga Orchestrator service issues commands to each participant and explicitly tracks state transitions and compensations.	Explicit, testable, centrally observable — but the orchestrator becomes a critical, must-be-highly-available component.

Real-Time Usage in Production

- **E-commerce checkout:** Order, Payment, Inventory, and Shipping services are coordinated in one saga, with compensations firing automatically on payment decline or out-of-stock so no service is left holding a half-completed transaction.
- **Travel booking (flight + hotel + car):** an orchestrated saga books all three legs and rolls back (cancels) whichever legs already succeeded if any one leg fails — a classic all-or-nothing saga across independently-owned third-party systems.
- **Loan origination in banking:** credit check, KYC verification, and fund disbursement run as saga steps; if disbursement fails after KYC passes, a compensating transaction reverses the credit hold instead of leaving the application stuck.

4.1 Choreography-Based Saga



OrderService.java (choreography participant)

```java
@Service
public class OrderService {
    private final KafkaTemplate<String, Object> kafka;
    private final OrderRepository repo;

    public void createOrder(Order order) {
        order.setStatus("PENDING_CREDIT");
        repo.save(order);
        kafka.send("order-events", order.getId().toString(),
            new OrderCreatedEvent(order.getId(), order.getCustomerId(), order.getTotalAmount()));
    }

    @KafkaListener(topics = "customer-events", groupId = "order-service")
    public void onCustomerEvent(CustomerEvent event) {
        Order order = repo.findById(event.orderId()).orElseThrow();
        if (event instanceof CreditReservedEvent) {
            order.setStatus("PENDING_STOCK");
            repo.save(order);
            kafka.send("stock-commands", order.getId().toString(),
                new ReserveStockCommand(order.getId(), order.getItems()));
        } else if (event instanceof CreditLimitExceededEvent) {
            order.setStatus("REJECTED");   // compensating: nothing to undo yet, just fail fast
            repo.save(order);
        }
    }

    @KafkaListener(topics = "product-events", groupId = "order-service")
    public void onProductEvent(ProductEvent event) {
        Order order = repo.findById(event.orderId()).orElseThrow();
        if (event instanceof StockReservedEvent) {
            order.setStatus("CONFIRMED");
            repo.save(order);
        } else if (event instanceof OutOfStockEvent) {
            order.setStatus("CANCELLED");
            repo.save(order);
            // COMPENSATING TRANSACTION: tell customer-service to release the credit hold
            kafka.send("customer-commands", order.getId().toString(),
                new ReleaseCreditCommand(order.getId(), order.getCustomerId(), order.getTotalAmount()));
        }
    }
}
```

CustomerService.java (participant + compensation handler)

```
@Service
public class CustomerService {
    @KafkaListener(topics = "order-events", groupId = "customer-service")
    public void onOrderCreated(OrderCreatedEvent event) {
        boolean ok = creditService.tryReserve(event.customerId(), event.totalAmount());
        if (ok)
            kafka.send("customer-events", event.orderId().toString(),
                new CreditReservedEvent(event.orderId(), event.customerId(), event.totalAmount()));
        else
            kafka.send("customer-events", event.orderId().toString(),
                new CreditLimitExceededEvent(event.orderId(), event.customerId()));
    }

    @KafkaListener(topics = "customer-commands", groupId = "customer-service")
    public void onReleaseCredit(ReleaseCreditCommand cmd) {
        creditService.release(cmd.customerId(), cmd.amount()); // undo the hold
    }
}
```

Choreography gotcha: With five or more steps, tracing "why did this order end up CANCELLED" means grep-ing through three services' logs correlated by orderId. This is the #1 reason teams migrate to orchestration once a saga passes ~3 participants.

4.2 Orchestration-Based Saga

Here a dedicated `OrderSagaOrchestrator` owns the state machine explicitly and issues commands (not just reacts to events), making the entire flow readable in one file.

OrderSagaState.java (persisted state machine)

```
@Entity
public class OrderSagaState {
    @Id private UUID sagaId;
    @Enumerated(EnumType.STRING)
    private SagaStep currentStep; // STARTED, CREDIT_RESERVED, STOCK_RESERVED, COMPLETED, COMPENSATING, FAILED
    private UUID orderId;
    private UUID customerId;
    private BigDecimal amount;
}
```

OrderSagaOrchestrator.java

```
@Service
public class OrderSagaOrchestrator {
    private final KafkaTemplate<String, Object> kafka;
    private final OrderSagaStateRepository sagaRepo;

    public void start(UUID orderId, UUID customerId, BigDecimal amount) {
        var state = new OrderSagaState(UUID.randomUUID(), SagaStep.STARTED, orderId, customerId, amount);
        sagaRepo.save(state);
        kafka.send("customer-commands", orderId.toString(),
            new ReserveCreditCommand(orderId, customerId, amount));
    }

    @KafkaListener(topics = "customer-events", groupId = "saga-orchestrator")
    public void onCreditResult(CustomerEvent event) {
        OrderSagaState state = sagaRepo.findByOrderId(event.orderId());
        if (event instanceof CreditReservedEvent) {
            state.setCurrentStep(SagaStep.CREDIT_RESERVED);
            sagaRepo.save(state);
            kafka.send("product-commands", state.getOrderId().toString(),
                new ReserveStockCommand(state.getOrderId(), null));
        } else {
            state.setCurrentStep(SagaStep.FAILED);
            sagaRepo.save(state);
            kafka.send("order-commands", state.getOrderId().toString(),
                new RejectOrderCommand(state.getOrderId(), "Insufficient credit"));
        }
    }

    @KafkaListener(topics = "product-events", groupId = "saga-orchestrator")
    public void onStockResult(ProductEvent event) {
        OrderSagaState state = sagaRepo.findByOrderId(event.orderId());
        if (event instanceof StockReservedEvent) {
            state.setCurrentStep(SagaStep.COMPLETED);
            sagaRepo.save(state);
            kafka.send("order-commands", state.getOrderId().toString(),
                new ConfirmOrderCommand(state.getOrderId()));
        } else { // StockUnavailable -- compensate the credit hold
            state.setCurrentStep(SagaStep.COMPENSATING);
            sagaRepo.save(state);
            kafka.send("customer-commands", state.getOrderId().toString(),
                new ReleaseCreditCommand(state.getOrderId(), state.getCustomerId(), state.getAmount()));
```

```
            kafka.send("order-commands", state.getOrderId().toString(),
                new RejectOrderCommand(state.getOrderId(), "Out of stock"));
        }
    }
}
```

Testing a Saga Failure Path

```
# Simulate the "insufficient credit" branch by creating a customer with a low limit,
# then placing an order that exceeds it:
curl -X POST http://localhost:8082/orders -d '{"customerId":"low-limit-cust","totalAmount":99999}'

# Poll the saga state table to watch the transition:
curl http://localhost:8084/sagas/order/{orderId}
# Expect: STARTED -> FAILED  (order rejected, no stock command ever issued)

# Simulate the "out of stock" branch (credit ok, stock fails) to see compensation fire:
curl -X POST http://localhost:8082/orders -d '{"customerId":"good-cust","items":[{"productId":"out-of-stock-sku"}]}'
# Expect saga: STARTED -> CREDIT_RESERVED -> COMPENSATING -> FAILED,
# and confirm via customer-service logs that the credit hold was released.
```

Gotchas across both styles:

- Compensating transactions are *not* a rollback — they are a new, forward-moving business transaction (e.g., "issue a refund," not "pretend the charge never happened"). Design them as first-class operations with their own idempotency guarantees.
- Every step and every compensation MUST be idempotent — Kafka's at-least-once delivery means `ReserveStockCommand` can arrive twice; use a dedup table keyed by `(orderId, step)`.
- Sagas can partially fail mid-compensation too (e.g., the credit-release message itself is lost) — production systems need a saga timeout/reaper job that re-drives stuck sagas.

When to use: Any multi-service business transaction that would have been a single ACID transaction in a monolith. Prefer orchestration once you have more than 2-3 participating services or need strong observability into the transaction's state.

Pattern 5 — Backend for Frontend (BFF) Client-Specific Aggregation

The Problem

A mobile app needs a lean payload (avatar thumbnail, last 3 orders); a web dashboard needs the full order history with pagination and analytics fields. Forcing both clients through one generic "orchestration API" leads to either over-fetching (mobile pays the bandwidth cost of desktop fields) or a bloated API riddled with conditional fields. BFF solves this with a thin aggregation layer *per client type*, each shaped exactly around that client's needs.

Real-Time Usage in Production

- **Streaming platforms (Netflix-style):** separate BFFs serve the mobile app, smart-TV app, and web app, each shaping the same catalog/recommendation data differently for very different device capabilities and bandwidth budgets.
- **Banking apps:** the mobile BFF returns a lightweight account summary for quick balance checks, while the web internet-banking BFF aggregates full transaction history, statements, and spend analytics in one call.
- **E-commerce apps:** the mobile BFF trims image sizes and drops non-essential fields to save data, while the web BFF adds recommendation widgets and richer product comparison data.

Architecture

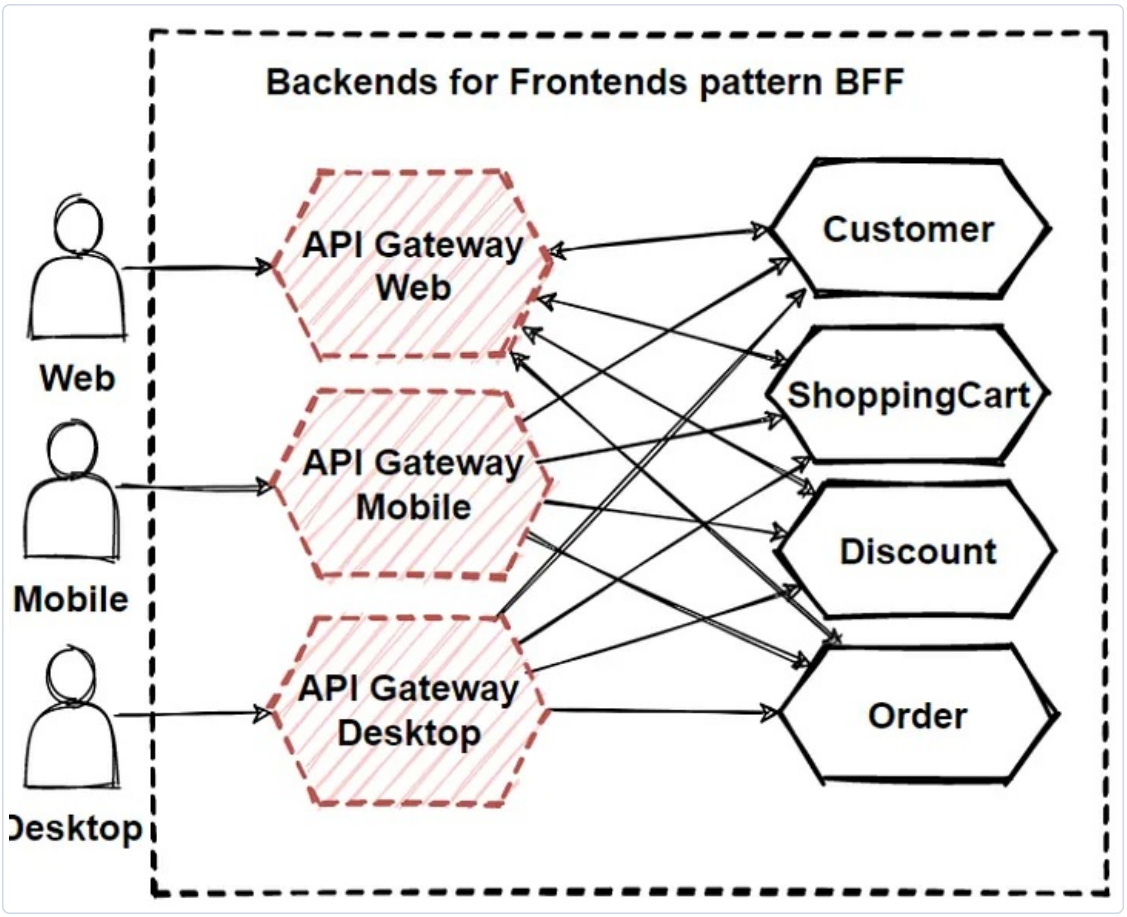


Fig. 7 — Three distinct BFFs (Web, Mobile, Desktop) each front the same set of backend services (Customer, ShoppingCart, Discount, Order), fanning out to whichever combination each client type actually needs.

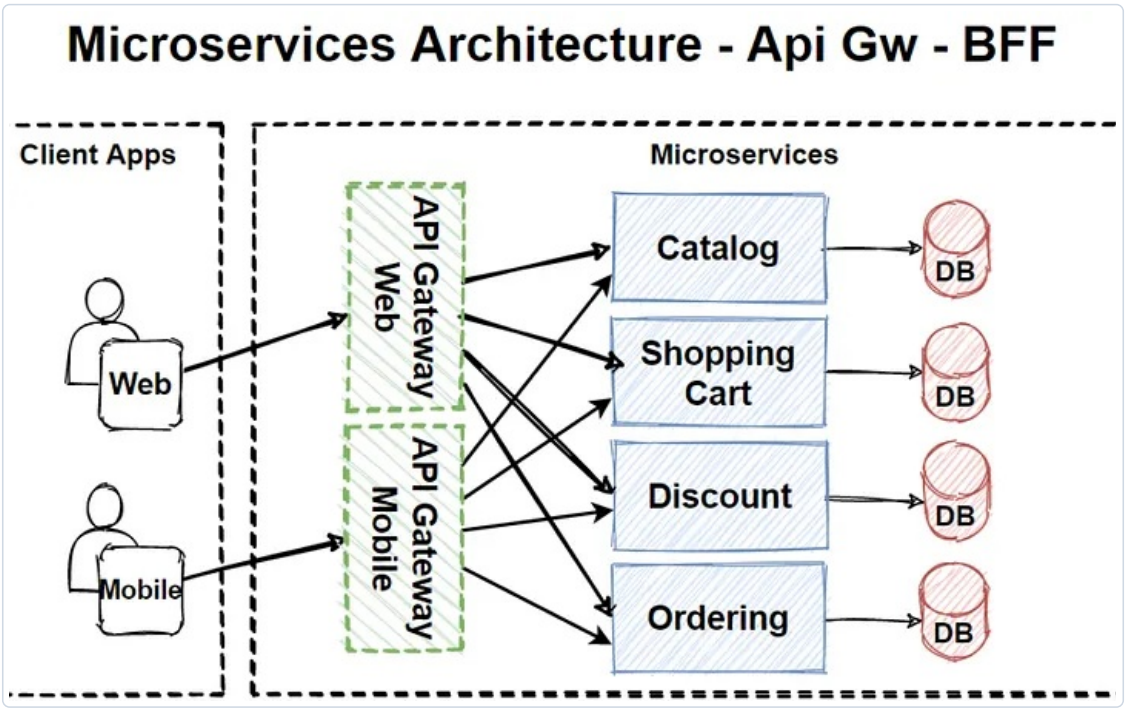


Fig. 8 — BFF layered behind an API Gateway per client type: a Web API Gateway and a Mobile API Gateway both route into the same Catalog, Shopping Cart, Discount, and Ordering services.

```
Mobile App ──► Mobile-BFF ─┐
                           ├──► User Service   (REST)
Web App    ──► Web-BFF    ─┤
                           ├──► Order Service  (REST)
                           └──► Product Service (REST)
Each BFF calls the same downstream services but shapes/aggregates the response differently.
```

Mobile BFF — lean, parallel fan-out with WebClient

pom.xml

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-webflux</artifactId>
</dependency>
```

MobileBffController.java — fans out concurrently, not sequentially

```
@RestController
@RequestMapping("/mobile-bff")
public class MobileBffController {
    private final WebClient userClient;
    private final WebClient orderClient;

    @GetMapping("/home/{userId}")
    public Mono<MobileHomeView> home(@PathVariable String userId) {
        Mono<UserSummary> userMono = userClient.get().uri("/users/{id}/summary", userId)
            .retrieve().bodyToMono(UserSummary.class)
            .timeout(Duration.ofMillis(800))
            .onErrorReturn(UserSummary.unavailable());

        Mono<List<OrderSummary>> ordersMono = orderClient.get()
            .uri("/orders/query/customer/{id}?limit=3", userId)
            .retrieve().bodyToFlux(OrderSummary.class).collectList()
            .timeout(Duration.ofMillis(800))
            .onErrorReturn(List.of());

        // Mono.zip fires both calls in parallel — critical for mobile latency budgets
        return Mono.zip(userMono, ordersMono)
            .map(tuple -> new MobileHomeView(tuple.getT1(), tuple.getT2()));
    }
}

public record MobileHomeView(UserSummary user, List<OrderSummary> recentOrders) {}
```

Web BFF — richer payload, server-side pagination pass-through

```
@RestController
@RequestMapping("/web-bff")
public class WebBffController {
    @GetMapping("/dashboard/{userId}")
    public Mono<WebDashboardView> dashboard(
            @PathVariable String userId,
            @RequestParam(defaultValue = "0") int page,
            @RequestParam(defaultValue = "20") int size) {
        Mono<UserFullProfile> userMono = userClient.get().uri("/users/{id}", userId)
            .retrieve().bodyToMono(UserFullProfile.class);
        Mono<Page<OrderDetail>> ordersMono = orderClient.get()
            .uri("/orders/query/customer/{id}?page={p}&size={s}", userId, page, size)
            .retrieve().bodyToMono(new ParameterizedTypeReference<Page<OrderDetail>>() {});
        Mono<SpendingAnalytics> analyticsMono = orderClient.get()
            .uri("/orders/analytics/customer/{id}", userId)
            .retrieve().bodyToMono(SpendingAnalytics.class);

        return Mono.zip(userMono, ordersMono, analyticsMono)
            .map(t -> new WebDashboardView(t.getT1(), t.getT2(), t.getT3()));
    }
}
```

Gotchas:

- Never let a BFF grow real business logic — it aggregates and reshapes; validation and domain rules stay in the owning service. A BFF with business logic is just an undocumented second copy of your domain.
- Always set per-call timeouts and fallbacks (as above) — one BFF endpoint fanning out to 4 services means a single slow downstream can drag down every client using that BFF.
- Duplicating a BFF per client type (mobile/web/partner-API) is the point, not a smell — but watch for three BFFs converging on identical aggregation logic, which signals it should move into a shared library or back into the owning service.

When to use: Multiple client types (mobile, web, third-party partners) with materially different data/latency needs consuming the same backend services.

Pattern 6 — API Gateway Single Entry Point

The Problem

Without a gateway, every client must know the network location of every microservice, and every service must independently implement auth, rate limiting, and CORS. The API Gateway centralizes these cross-cutting concerns and gives clients one stable entry point regardless of how services are split, merged, or moved.

Conceptually, the API Gateway is the distributed-systems equivalent of the Facade pattern from object-oriented design: just as a facade class hides a complex subsystem behind one simplified interface, an API Gateway sits as a reverse proxy between client applications and the internal microservices, exposing a single entry point and encapsulating the underlying service topology entirely. Clients depend on the gateway's stable contract, never on the internal services directly — so services can be split, merged, rewritten, or relocated behind the gateway without any client-side change.

Main Features of the API Gateway Pattern

Because the gateway sits in front of every request, it's the natural, centralized place to implement a handful of recurring concerns rather than duplicating them in every microservice:

1. Reverse Proxy / Gateway Routing

The gateway's core job is routing: it redirects incoming requests to the correct internal microservice endpoint, typically using layer-7 (HTTP-aware) routing rules. This decouples client applications from the internal services at the network level — clients only ever address the gateway. It also abstracts internal topology changes away from clients entirely: a backend service can be renamed, re-versioned, or moved to a new host, and as long as the gateway's routing rules are updated, no client needs to change anything.

2. Request Aggregation

The gateway can combine several internal microservice calls into a single client-facing request/response — the client sends one request, the gateway fans out to multiple backend services, aggregates their results, and returns one combined response. This is the same technique built out in Part I's Service Aggregator pattern (Bonus A), and its main payoff is reducing chatty round-trips between client applications and the backend.

3. Cross-Cutting Concerns & Gateway Offloading

Since the gateway already handles every request in one centralized place, it's the natural place to implement functionality that would otherwise need to be duplicated in every microservice:

- Authentication and authorization
- Service discovery integration
- Response caching
- Retry policies, circuit breaking, and quality-of-service controls
- Rate limiting and throttling
- Load balancing
- Logging, tracing, and correlation-id propagation
- Header, query-string, and claims transformation
- IP allowlisting

> **Gateway product feature sets look remarkably similar across ecosystems.** This guide builds its gateway on Spring Cloud Gateway (Java), but the same feature set shows up in every major API gateway product — for example, **Ocelot**, the equivalent gateway commonly used in .NET microservice stacks:

Ocelot Features	
Routing	Authentication
Request Aggregation	Authorization
Service Discovery with Consul & Eureka	Throttling
Load Balancing	Logging, Tracing
Correlation Pass-Through	Headers/Query String Transformation
Quality of Service	Custom Middleware

Fig. 8a — Ocelot's feature set — Routing, Request Aggregation, Service Discovery (Consul & Eureka), Load Balancing, Correlation Pass-Through, Quality of Service, Authentication, Authorization, Throttling, Logging/Tracing, Header/Query-String Transformation, and Custom Middleware — maps almost one-to-one onto the feature list above, regardless of which language or gateway product a team standardizes on.

Real-Time Usage in Production

- **Streaming/large-scale platforms:** Netflix's original Zuul gateway (and its Spring Cloud Gateway successors elsewhere) centralizes routing across hundreds of backend services behind one entry point for all client apps.
- **Banking platforms:** the gateway enforces OAuth2/JWT authentication and per-client rate limits before any request reaches core banking services, keeping auth logic out of every individual service.
- **SaaS platforms:** gateways handle API versioning, canary/blue-green traffic splitting, and request/response transformation centrally, so individual services don't each reimplement it.

Architecture

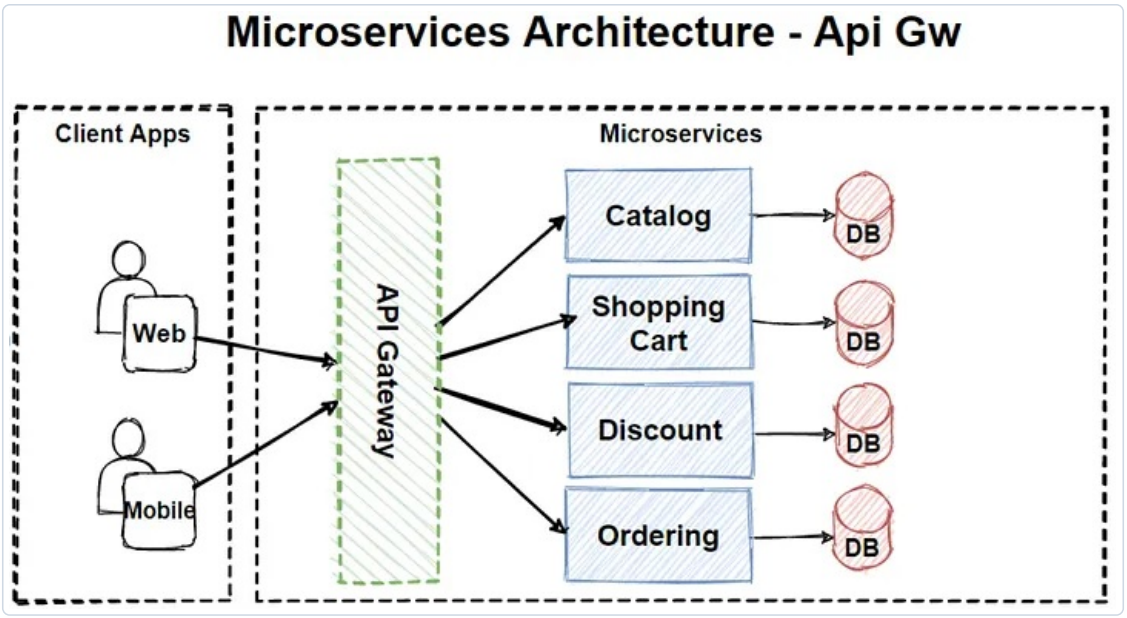


Fig. 9 — Web and Mobile clients both enter through a single API Gateway, which routes to Catalog, Shopping Cart, Discount, and Ordering — each still owning its own database.

```
Client ──► API Gateway (Spring Cloud Gateway :8080)
               │  routes by path predicate
               ├─ /users/**     ──► user-service:8085
               ├─ /orders/**    ──► order-service:8082
               ├─ /products/** ──► product-service:8081
               └─ cross-cutting: JWT validation, rate limiting, request logging, CORS
```

pom.xml

```
<dependency>
  <groupId>org.springframework.cloud</groupId>
  <artifactId>spring-cloud-starter-gateway</artifactId>
</dependency>
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-oauth2-resource-server</artifactId>
</dependency>
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-data-redis-reactive</artifactId>
</dependency>
```

application.yml — declarative routes + rate limiting + auth

```
server:
  port: 8080
spring:
  cloud:
    gateway:
      routes:
        - id: user-service
          uri: lb://user-service
          predicates: [ Path=/users/** ]
          filters:
            - name: RequestRateLimiter
              args:
                redis-rate-limiter.replenishRate: 20
                redis-rate-limiter.burstCapacity: 40
        - id: order-service
          uri: lb://order-service
          predicates: [ Path=/orders/** ]
          filters:
            - StripPrefix=0
            - AddRequestHeader=X-Gateway-Source, api-gateway
        - id: product-service
          uri: lb://product-service
          predicates: [ Path=/products/** ]
  security:
    oauth2:
      resourceserver:
        jwt:
          issuer-uri: https://auth.example.com/realms/ecommerce
```

SecurityConfig.java — enforce JWT on everything except public routes

```
@Configuration
@EnableWebFluxSecurity
```

```
public class SecurityConfig {
    @Bean
    public SecurityWebFilterChain filterChain(ServerHttpSecurity http) {
        return http
            .authorizeExchange(ex -> ex
                .pathMatchers("/products/**").permitAll()    // public catalog browsing
                .pathMatchers("/orders/**", "/users/**").authenticated()
                .anyExchange().denyAll())
            .oauth2ResourceServer(oauth -> oauth.jwt(Customizer.withDefaults()))
            .csrf(ServerHttpSecurity.CsrfSpec::disable)
            .build();
    }
}
```

GlobalLoggingFilter.java — custom cross-cutting filter

```
@Component
public class GlobalLoggingFilter implements GlobalFilter, Ordered {
    private static final Logger log = LoggerFactory.getLogger(GlobalLoggingFilter.class);

    @Override
    public Mono<Void> filter(ServerWebExchange exchange, GatewayFilterChain chain) {
        long start = System.currentTimeMillis();
        String path = exchange.getRequest().getURI().getPath();
        return chain.filter(exchange).doFinally(signal ->
            log.info("{} completed in {}ms [{}]", path, System.currentTimeMillis() - start,
                    exchange.getResponse().getStatusCode()));
    }

    @Override
    public int getOrder() { return -1; }
}
```

Testing Routing, Auth, and Rate Limiting

```
# public route, no token needed
curl http://localhost:8080/products/123

# protected route without token -> 401
curl -i http://localhost:8080/orders/query/customer/abc

# protected route with a valid bearer token -> 200
curl http://localhost:8080/orders/query/customer/abc -H "Authorization: Bearer $JWT"

# hammer the rate limiter (expect 429 after burst capacity of 40 is exceeded)
for i in $(seq 1 60); do curl -s -o /dev/null -w "%{http_code}\n" http://localhost:8080/users/1; done
```

Gotchas:

- The gateway becomes a single point of failure by definition — it must be deployed with multiple replicas behind a load balancer, never as a single instance.
- Avoid putting business logic in gateway filters — routing, auth, rate limiting, and logging only; anything domain-specific belongs in the owning service or a BFF.
- Watch gateway timeout defaults — a slow downstream service without an explicit timeout can exhaust the gateway's connection pool and take down routing for every other service.

When to use: Virtually every microservice system with external clients needs exactly one gateway layer. Internal service-to-service calls should bypass the gateway (via service discovery) to avoid unnecessary latency.

Pattern 7 — Strangler Fig Pattern Incremental Migration

The Problem

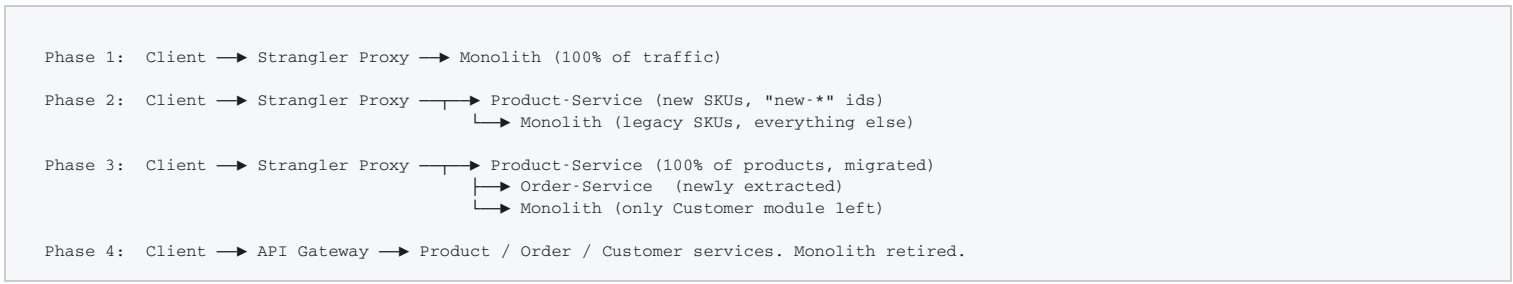
Rewriting a legacy monolith in one "big bang" cutover is high-risk and usually fails. The Strangler Fig pattern (named after the vine that grows around a host tree and gradually replaces it) puts a routing facade in front of the monolith, then migrates functionality piece by piece: new requests for a migrated capability go to the new microservice, everything else still goes to the monolith — until eventually nothing is left routing to the old system and it is decommissioned.

Real-Time Usage in Production

- **Large enterprise/legacy modernization:** banks and insurers moving decades-old mainframe/COBOL systems to microservices route new transactions to the new services while old ones still flow to the legacy core, until every capability is migrated.
- **E-commerce platform rewrites:** a legacy PHP or monolithic Java application is migrated module by module (catalog first, then checkout, then customer accounts) behind a strangler proxy, with production traffic gradually shifted per module.

- **Government / insurance ERP modernization:** old case-management or policy systems are strangled incrementally, letting teams verify each newly extracted service under real production load before fully retiring the old code path.

Architecture — Migration Timeline



Implementation — Facade with Feature-Flag-Driven Routing

pom.xml

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-web</artifactId>
</dependency>
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-actuator</artifactId>
</dependency>
```

StranglerProxyController.java — richer routing than a hard-coded prefix check

```
@RestController
public class StranglerProxyController {
    private final RestClient restClient;
    private final MigrationRegistry migrationRegistry; // backed by a feature-flag service or DB table

    @GetMapping("/products/{id}")
    public ResponseEntity<Product> getProduct(@PathVariable String id) {
        if (migrationRegistry.isMigrated("product", id)) {
            return restClient.get().uri("http://product-service/products/{id}", id)
                .retrieve().toEntity(Product.class);
        }
        return restClient.get().uri("http://monolith/legacy/products/{id}", id)
            .retrieve().toEntity(Product.class);
    }

    @PostMapping("/products")
    public ResponseEntity<Product> createProduct(@RequestBody Product product) {
        // new writes always go to the new service once migration for "create" has begun --
        // this is the "stop the bleeding" step: no new data enters the legacy schema
        if (migrationRegistry.isWriteMigrated("product")) {
            return restClient.post().uri("http://product-service/products")
                .body(product).retrieve().toEntity(Product.class);
        }
        return restClient.post().uri("http://monolith/legacy/products")
            .body(product).retrieve().toEntity(Product.class);
    }
}
```

MigrationRegistry.java — the routing decision engine

```
@Component
public class MigrationRegistry {
    private final MigrationFlagRepository flags; // simple Postgres table: capability, cutover_percent, migrated_ids

    public boolean isMigrated(String capability, String entityId) {
        MigrationFlag flag = flags.findByCapability(capability);
        if (flag.getCutoverPercent() >= 100) return true;
        if (flag.getMigratedIdPrefixes().stream().anyMatch(entityId::startsWith)) return true;
        // gradual rollout by consistent hashing on entity id, e.g. 25% cutover
        return Math.abs(entityId.hashCode() % 100) < flag.getCutoverPercent();
    }

    public boolean isWriteMigrated(String capability) {
        return flags.findByCapability(capability).isWritesMigrated();
    }
}
```

Dual-Write / Sync Strategy During Migration

During the overlap window, data created in the legacy monolith must still be visible to the new service (and vice-versa). The two common

approaches:

- **Change Data Capture (CDC):** tail the monolith's database write-ahead log (Debezium) and publish changes as Kafka events for the new service to consume — no application code changes needed in the legacy system.
- **Dual write from the proxy:** the strangler proxy writes to both systems during the transition (higher risk of drift, needs reconciliation jobs, but requires no CDC infrastructure).

Debezium connector config (CDC approach — recommended)

```
{
  "name": "monolith-product-connector",
  "config": {
    "connector.class": "io.debezium.connector.postgresql.PostgresConnector",
    "database.hostname": "monolith-db",
    "database.dbname": "monolith",
    "table.include.list": "public.products",
    "topic.prefix": "monolith-cdc",
    "plugin.name": "pgoutput"
  }
}
```

Testing a Gradual Cutover

```
# set product migration to 25% cutover
curl -X POST http://localhost:8090/admin/migration-flags \
  -d '{"capability":"product","cutoverPercent":25,"writesMigrated":false}'

# hit the same endpoint many times with different ids -- roughly 1/4 should route to product-service
for id in p1 p2 p3 p4 p5 p6 p7 p8; do curl -s http://localhost:8090/products/$id -o /dev/null -w "$id -> %{http_code}\n"; done

# check gateway/proxy logs to confirm the routing split matches the configured percentage
```

Gotchas:

- Never let the migration flag logic live only in code deployed with the service — put cutover percentages in a database/feature-flag service you can change without a redeploy, since rollback speed during migration incidents matters enormously.
- Data written during the overlap period in both systems needs a reconciliation job; assume drift will happen and build a diff-and-repair batch job before you rely on dual writes.
- Don't declare "migration complete" until you've deleted (not just stopped routing to) the legacy code path — dead routes accumulate and become a security liability.

When to use: Migrating any non-trivial legacy monolith to microservices. Avoid the big-bang rewrite; strangle it capability by capability instead.

Pattern 8 — Circuit Breaker Fault Isolation

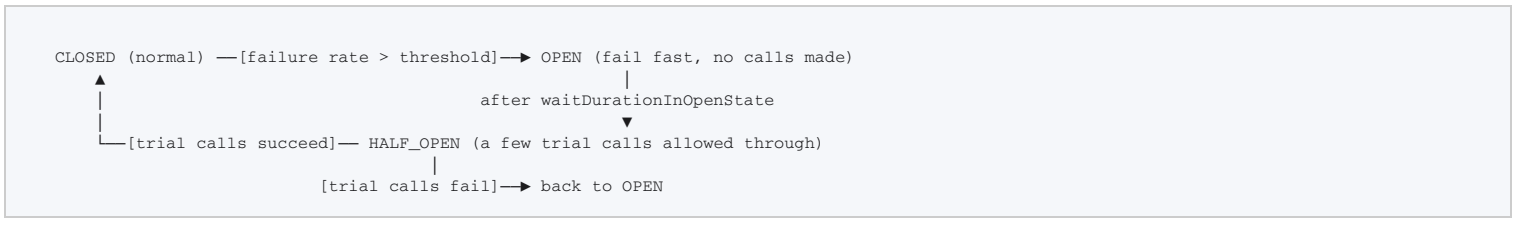
The Problem

When Order Service calls a struggling Product Service, every blocked thread waiting on a slow response consumes a connection/thread. Under sustained failure, this cascades: Order Service itself becomes unresponsive to its own callers, and the outage spreads outward. The Circuit Breaker pattern stops calling a failing dependency once a failure threshold is crossed, fails fast with a fallback, and periodically probes to see if the dependency has recovered.

Real-Time Usage in Production

- **Netflix (Hystrix's origin):** circuit breakers wrap thousands of inter-service calls across the platform specifically to stop a single slow dependency from cascading into a platform-wide outage.
- **Payment gateways:** calls to third-party processors (Stripe/PayPal/Razorpay) are wrapped in a circuit breaker so a provider outage triggers an instant fallback (queue-and-retry-later message) instead of hanging checkout requests.
- **Ride-sharing / logistics apps:** calls to maps/geolocation providers are circuit-broken so a mapping API outage degrades gracefully (last-known-location fallback) instead of blocking trip booking entirely.

The Three States



pom.xml

```
<dependency>
  <groupId>io.github.resilience4j</groupId>
  <artifactId>resilience4j-spring-boot3</artifactId>
  <version>${resilience4j.version}</version>
</dependency>
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-aop</artifactId>
</dependency>
```

application.yml — circuit breaker + retry + bulkhead stacked together

```
resilience4j:
  circuitbreaker:
    instances:
      productService:
        registerHealthIndicator: true
        slidingWindowType: COUNT_BASED
        slidingWindowSize: 20
        minimumNumberOfCalls: 10
        failureRateThreshold: 50
        waitDurationInOpenState: 15000
        permittedNumberOfCallsInHalfOpenState: 3
        automaticTransitionFromOpenToHalfOpenEnabled: true
  retry:
    instances:
      productService:
        maxAttempts: 3
        waitDuration: 200
        retryExceptions:
          - java.io.IOException
          - org.springframework.web.client.ResourceAccessException
  bulkhead:
    instances:
      productService:
        maxConcurrentCalls: 15
```

OrderClient.java — stacking Retry inside CircuitBreaker inside Bulkhead

```
@Service
public class OrderClient {
    private final RestClient restClient;
    private static final Logger log = LoggerFactory.getLogger(OrderClient.class);

    @Bulkhead(name = "productService")
    @Retry(name = "productService")
    @CircuitBreaker(name = "productService", fallbackMethod = "fallback")
    public Product getProduct(String productId) {
        return restClient.get().uri("http://product-service/products/{id}", productId)
            .retrieve().body(Product.class);
    }

    // fallback signature MUST match + accept the Throwable as the last parameter
    public Product fallback(String productId, Throwable t) {
        log.warn("Product service unavailable for {}, serving cached fallback: {}", productId, t.toString());
        return productCache.getCached(productId)
            .orElse(Product.placeholder(productId, "Temporarily unavailable"));
    }
}
```

Programmatic usage + event listeners for observability

```
@Component
public class CircuitBreakerMonitor {
    public CircuitBreakerMonitor(CircuitBreakerRegistry registry) {
        registry.circuitBreaker("productService").getEventPublisher()
            .onStateTransition(event ->
                log.info("CircuitBreaker '{}' state: {} -> {}",
                    event.getCircuitBreakerName(),
                    event.getStateTransition().getFromState(),
                    event.getStateTransition().getToState()));
    }
}
```

Testing State Transitions

```
# simulate product-service being down (stop the container)
docker stop product-service

# hammer the order-service endpoint that depends on it
for i in $(seq 1 25); do curl -s -o /dev/null -w "%{http_code} " http://localhost:8082/orders/with-product/abc; done
# first ~10 calls: slow failures (CLOSED, counting toward the threshold)
```

```
# after threshold crossed: instant 200s with fallback payload (OPEN, fail-fast)

# check breaker state via actuator
curl http://localhost:8082/actuator/circuitbreakers | jq '.circuitBreakers.productService.state'

# bring product-service back up, wait 15s (waitDurationInOpenState), then retry
docker start product-service
sleep 16
curl http://localhost:8082/orders/with-product/abc   # HALF_OPEN trial call, should succeed and re-CLOSE
```

Gotchas:

- A fallback that silently returns fake success data (e.g., "assume stock available") can be worse than the original failure — always make fallback behavior visibly degraded (cached-stale flag, "please retry" messaging) so downstream code and users know they're in a degraded state.
- Setting `slidingWindowSize` too small makes the breaker trip on noise; too large delays detection of a real outage. Tune against your actual call volume per instance.
- Stacking Retry + CircuitBreaker in the wrong order can retry against an already-open breaker pointlessly, or worse, retry storms can themselves cause the cascading failure the pattern is meant to prevent — always put Retry inside CircuitBreaker (as annotated above, innermost to outermost: CircuitBreaker → Retry → Bulkhead in execution order since annotations apply outside-in).

When to use: Any synchronous call to another service or external dependency that could be slow or unavailable — which in practice means almost every inter-service REST/gRPC call in production.

Pattern 9 — Externalized Configuration Config as a Service

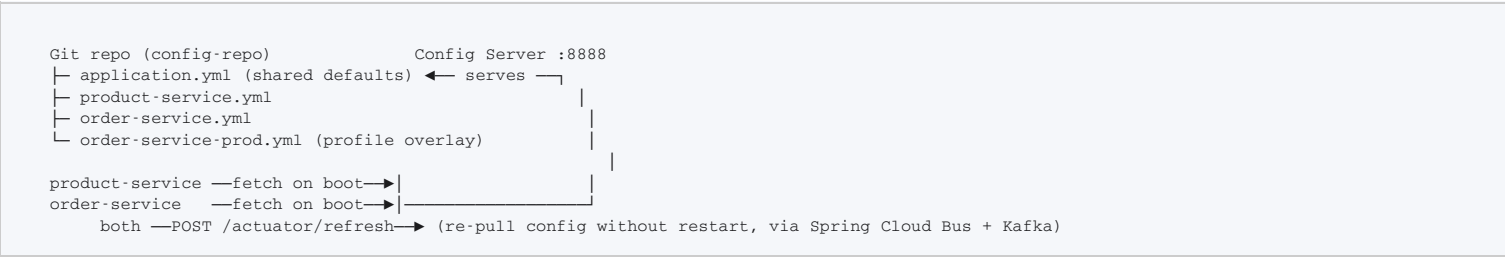
The Problem

Baking database URLs, feature flags, and connection pool sizes into each service's packaged JAR means every config change requires a full rebuild and redeploy — even for something as trivial as tweaking a timeout. Externalized Configuration moves all environment-specific settings into a central, versioned source (Git-backed Spring Cloud Config Server) that services fetch at startup and can refresh at runtime.

Real-Time Usage in Production

- **Multi-region deployments:** the same service binary runs differently per region — tax rules, currency formatting, feature flags — by pulling region-specific config at startup instead of building region-specific JARs.
- **Banking systems:** interest rates, transaction limits, and fraud-rule thresholds are externalized so business/compliance teams can adjust them without waiting on an engineering redeploy.
- **Multi-tenant SaaS platforms:** per-tenant configuration (branding, usage limits, enabled modules) is fetched at runtime, letting one codebase serve many customers with different settings.

Architecture



Config Server Setup

pom.xml (config-server module)

```
<dependency>
  <groupId>org.springframework.cloud</groupId>
  <artifactId>spring-cloud-config-server</artifactId>
</dependency>
<dependency>
  <groupId>org.springframework.cloud</groupId>
  <artifactId>spring-cloud-starter-bus-kafka</artifactId>
</dependency>
```

ConfigServerApplication.java

```
@SpringBootApplication
@EnableConfigServer
```

```java
public class ConfigServerApplication {
    public static void main(String[] args) { SpringApplication.run(ConfigServerApplication.class, args); }
}
```

application.yml (config-server itself)

```yaml
server:
  port: 8888
spring:
  cloud:
    config:
      server:
        git:
          uri: https://github.com/example-org/ecommerce-config-repo
          default-label: main
          clone-on-start: true
  kafka:
    bootstrap-servers: localhost:9092
```

config-repo/order-service.yml (fetched by order-service at startup)

```yaml
app:
  config:
    databaseUrl: jdbc:postgresql://postgres-order:5432/orderdb
    maxConnections: 50
    features:
      newCheckoutFlow: false
resilience4j:
  circuitbreaker:
    instances:
      productService:
        failureRateThreshold: 50
```

Client-Side Wiring

pom.xml (order-service)

```xml
<dependency>
  <groupId>org.springframework.cloud</groupId>
  <artifactId>spring-cloud-starter-config</artifactId>
</dependency>
<dependency>
  <groupId>org.springframework.cloud</groupId>
  <artifactId>spring-cloud-starter-bus-kafka</artifactId>
</dependency>
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-actuator</artifactId>
</dependency>
```

application.yml (order-service — points at config server)

```yaml
spring:
  application:
    name: order-service
  config:
    import: "configserver:http://localhost:8888"
  cloud:
    config:
      fail-fast: true
      retry:
        max-attempts: 5
management:
  endpoints:
    web:
      exposure:
        include: refresh, health, circuitbreakers
```

AppConfig.java — a live-refreshable config bean

```java
@Component
@ConfigurationProperties(prefix = "app.config")
@RefreshScope   // bean is re-created on /actuator/refresh or bus-refresh event
public class AppConfig {
    private String databaseUrl;
    private int maxConnections;
    private Map<String, Boolean> features = new HashMap<>();
    // getters/setters
}
```

FeatureFlagController.java — using the live flag

```java
@RestController
public class CheckoutController {
    private final AppConfig config; // @RefreshScope proxy injected here

    @PostMapping("/checkout")
    public CheckoutResponse checkout(@RequestBody CheckoutRequest req) {
        if (Boolean.TRUE.equals(config.getFeatures().get("newCheckoutFlow"))) {
            return newCheckoutService.process(req);
        }
        return legacyCheckoutService.process(req);
    }
}
```

Testing Live Refresh Without Redeploy

```bash
# 1. edit config-repo/order-service.yml -> set newCheckoutFlow: true, commit & push
# 2. broadcast the refresh to ALL instances via Spring Cloud Bus (no per-pod curl needed)
curl -X POST http://localhost:8888/actuator/busrefresh

# OR refresh a single instance directly:
curl -X POST http://localhost:8082/actuator/refresh

# 3. verify the new value took effect without a restart:
curl -X POST http://localhost:8082/checkout -d '{"cart":"..."}'   # now uses newCheckoutService
```

Gotchas:

- Secrets (DB passwords, API keys) should never sit in plaintext in the Git config repo — integrate with Vault or use Config Server's built-in encryption (`{cipher}...` values) instead.
- `fail-fast: true` means a Config Server outage prevents dependent services from even starting — run Config Server with HA replicas, and keep a local fallback profile for true disaster scenarios.
- `@RefreshScope` beans are recreated, not mutated — anything holding a stale reference to the old bean instance (e.g., captured in a background thread at startup) won't see the update. Re-inject via the proxy, don't cache the bean yourself.

When to use: Any multi-environment, multi-instance deployment where config needs to change independently of code deploys — which is effectively every production microservice system.

Pattern 10 — Consumer-Driven Contract Testing Pact

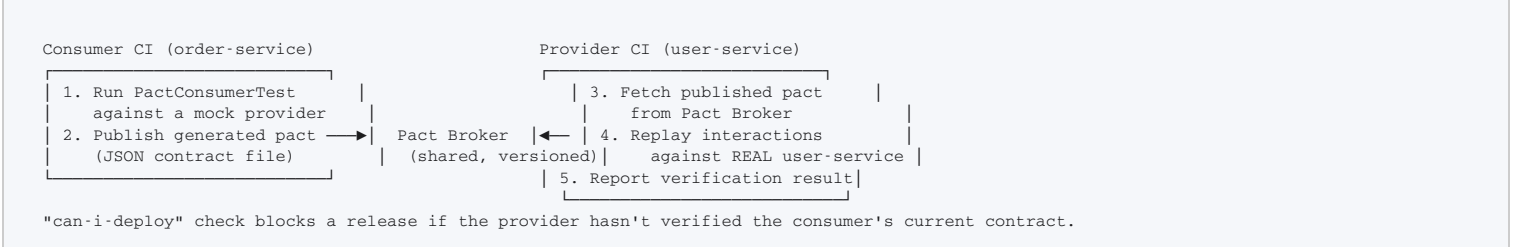
The Problem

Order Service depends on User Service's `/users/{id}` response shape. If User Service changes that shape in a way Order Service doesn't expect, you only find out in a shared staging environment — or worse, in production. Consumer-Driven Contract (CDC) testing has the *consumer* (Order Service) define exactly what it expects from the *provider* (User Service) as a "contract" file, which the provider then verifies against in its own CI pipeline — without either service needing the other running.

Real-Time Usage in Production

- **Large microservice organizations (PayPal/Atlassian-style):** Pact contracts run in every consumer and provider's CI pipeline to catch breaking API changes before they ever reach a shared staging environment.
- **Banking third-party integrations:** contract tests against partner/payment-network APIs ensure a partner's API version change doesn't silently break the bank's core consuming service.
- **Mobile app + backend teams:** contract testing catches a backend API shape change during CI, before it would otherwise only surface after a mobile app release has already gone to the app stores.

Architecture



Consumer Side (order-service) — define the expectation

pom.xml

```
<dependency>
  <groupId>au.com.dius.pact.consumer</groupId>
  <artifactId>junit5</artifactId>
  <version>4.6.5</version>
  <scope>test</scope>
</dependency>
```

UserServicePactTest.java

```
@ExtendWith(PactConsumerTestExt.class)
@PactTestFor(providerName = "UserService", pactVersion = PactSpecVersion.V3)
public class UserServicePactTest {

    @Pact(consumer = "OrderService")
    public RequestResponsePact validUserPact(PactDslWithProvider builder) {
        return builder
            .given("user with id 1 exists and is active")
            .uponReceiving("a request for an existing active user")
                .path("/users/1").method("GET")
            .willRespondWith()
                .status(200)
                .headers(Map.of("Content-Type", "application/json"))
                .body(new PactDslJsonBody()
                    .stringType("id", "1")
                    .stringType("name", "John Doe")
                    .stringType("status", "ACTIVE")
                    .numberType("creditLimit", 5000))
            .toPact();
    }

    @Test
    @PactTestFor(pactMethod = "validUserPact")
    void testFetchesActiveUser(MockServer mockServer) {
        UserClient client = new UserClient(mockServer.getUrl());
        UserDto user = client.fetchUser("1");
        assertThat(user.status()).isEqualTo("ACTIVE");
        assertThat(user.creditLimit()).isGreaterThan(BigDecimal.ZERO);
    }

    @Pact(consumer = "OrderService")
    public RequestResponsePact missingUserPact(PactDslWithProvider builder) {
        return builder
            .given("user with id 999 does not exist")
            .uponReceiving("a request for a nonexistent user")
                .path("/users/999").method("GET")
            .willRespondWith().status(404)
            .toPact();
    }
}
```

Publishing the contract in CI (order-service pipeline)

```
mvn test pact:publish \
  -Dpact.consumer.version=$(git rev-parse --short HEAD) \
  -Dpact.broker.url=https://pact-broker.example.com
```

Provider Side (user-service) — verify against the real implementation

pom.xml

```
<dependency>
  <groupId>au.com.dius.pact.provider</groupId>
  <artifactId>junit5</artifactId>
  <version>4.6.5</version>
  <scope>test</scope>
</dependency>
```

UserServiceProviderTest.java

```
@Provider("UserService")
@PactBroker(url = "https://pact-broker.example.com")
@SpringBootTest(webEnvironment = SpringBootTest.WebEnvironment.RANDOM_PORT)
public class UserServiceProviderTest {
    @LocalServerPort int port;

    @BeforeEach
    void setTarget(PactVerificationContext context) {
        context.setTarget(new HttpTestTarget("localhost", port));
```

```
    }

    @TestTemplate
    @ExtendWith(PactVerificationInvocationContextProvider.class)
    void verifyPact(PactVerificationContext context) { context.verifyInteraction(); }

    // "given()" clauses in the consumer pact map to @State methods that seed test data
    @State("user with id 1 exists and is active")
    void userExists() {
        userRepository.save(new User("1", "John Doe", Status.ACTIVE, BigDecimal.valueOf(5000)));
    }

    @State("user with id 999 does not exist")
    void userDoesNotExist() { userRepository.deleteById("999"); }
}
```

Gating Deploys with can-i-deploy

```
# in the deploy pipeline for user-service, BEFORE deploying to production:
pact-broker can-i-deploy \
  --pacticipant UserService --version $(git rev-parse --short HEAD) \
  --to-environment production
# exits non-zero (blocks deploy) if any active consumer's contract is unverified against this version
```

Gotchas:

- Contracts describe the shape and a few example values the *consumer actually depends on* — not every field the provider returns. Over-specifying values (exact timestamps, IDs) makes tests brittle; prefer matchers (`stringType` , `numberType`) over literal value assertions.
- `@State` setup methods must actually create the described state in the provider's test database — a missing or wrong state handler causes false-positive verifications.
- Contract tests are not a substitute for a handful of true end-to-end tests in staging — they catch shape/schema drift extremely well, but not cross-service business-logic bugs.

When to use: Any team boundary where one service's API changes could silently break another team's service — which is most inter-team microservice integrations. Combine with `can-i-deploy` gates in CI/CD for real safety, not just documentation value.

Bonus Patterns — Communication & Event-Driven Architecture

The diagrams below cover three additional patterns that show up constantly in real microservice codebases: an anti-pattern to recognize and avoid (Chain Queries), its fix (Service Aggregator), and the broader event-driven communication style (Message Broker / Publish-Subscribe) that most of the patterns in this guide are built on top of.

Bonus A — Chain Queries (Anti-Pattern) → Service Aggregator Pattern

Communication

The Problem: Chain Queries

A naive synchronous implementation of "add item to cart" has ShoppingCart call Product for price validation, then call Pricing for discount calculation, then Order needs to call Billing and Shipping in turn. Each hop adds latency, and a failure deep in the chain (say, Shipping) fails the entire top-level request even though Shipping isn't even relevant to adding an item to a cart.

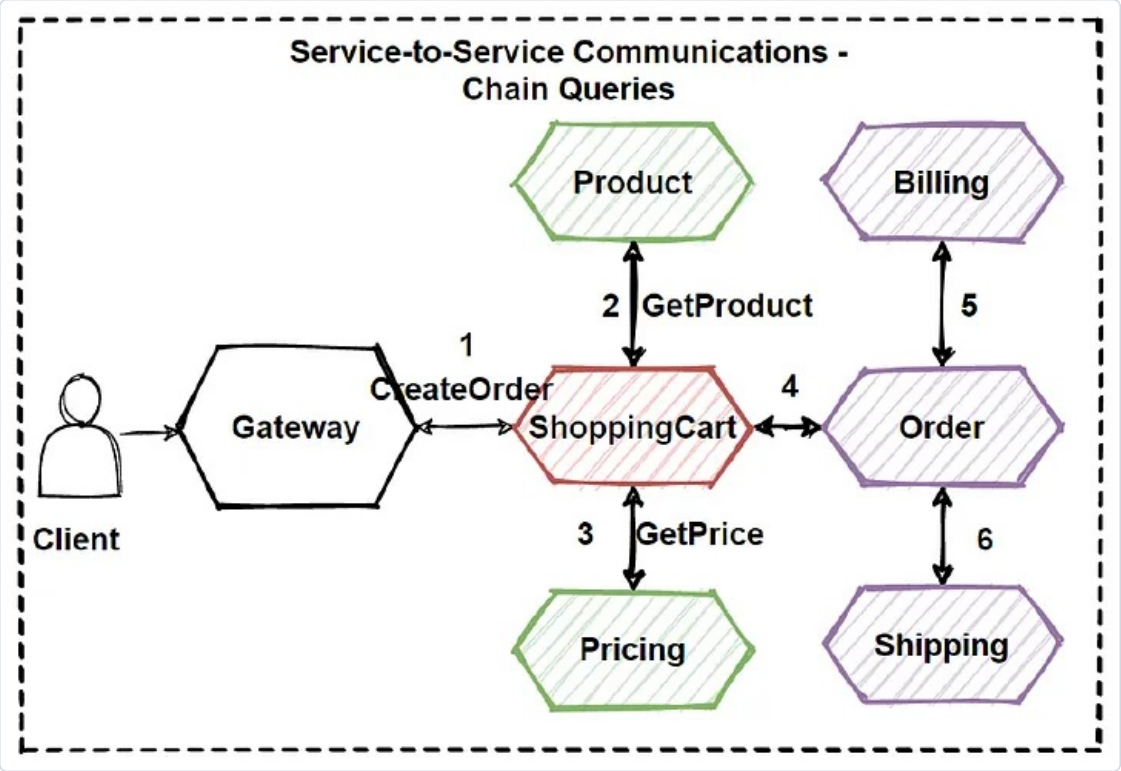


Fig. 10 — A single client CreateOrder call cascades through six sequential hops (Gateway → ShoppingCart → Product → Pricing → Order → Billing → Shipping). Total latency is the sum of every hop, and any one failure anywhere in the chain fails the whole request.

Why this hurts in production: latency compounds linearly with chain depth, a single slow or failing service anywhere in the chain takes down the whole request, and the calling code becomes tightly coupled to the exact sequence and existence of every downstream service.

The Fix: Service Aggregator Pattern

Instead of services calling each other in a deep chain, an Aggregator sits between the Gateway and the downstream services, fanning out calls in parallel and combining the results — the same idea as BFF (Pattern 5) but applied to a single operation rather than a whole client type.

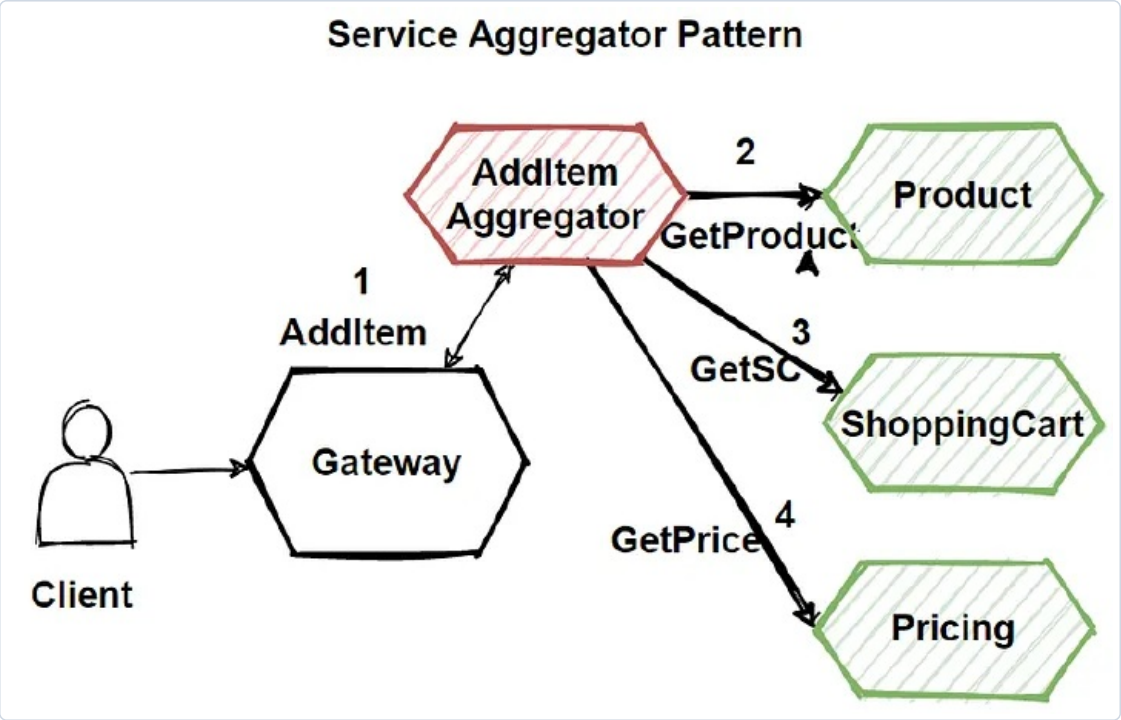


Fig. 11 — An AddItemAggregator receives the AddItem call once, then fans out to Product, ShoppingCart, and Pricing in parallel (steps 2, 3, 4 all originate from the aggregator, not from each other), instead of chaining through them sequentially.

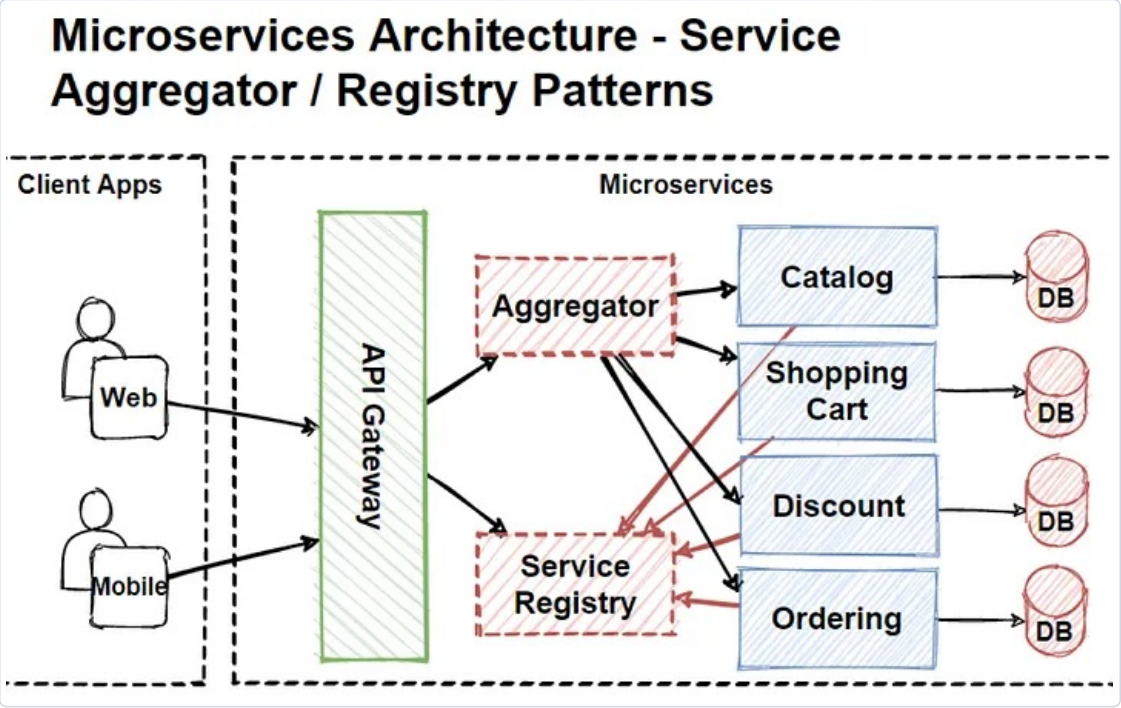


Fig. 12 — At platform scale, the Aggregator sits behind the API Gateway and consults a Service Registry to discover the current, healthy instances of Catalog, Shopping Cart, Discount, and Ordering before fanning out to them.

AddItemAggregator.java — parallel fan-out instead of a chain

```
@Service
public class AddItemAggregator {
    private final WebClient productClient;
    private final WebClient cartClient;
    private final WebClient pricingClient;

    public Mono<AddItemResult> addItem(AddItemRequest req) {
        Mono<ProductDto> productMono = productClient.get()
            .uri("/products/{id}", req.productId()).retrieve().bodyToMono(ProductDto.class);
        Mono<PriceDto> priceMono = pricingClient.get()
            .uri("/pricing/{id}", req.productId()).retrieve().bodyToMono(PriceDto.class);

        // both calls fire concurrently -- total latency = slowest single call, not the sum
        return Mono.zip(productMono, priceMono)
            .flatMap(tuple -> cartClient.post().uri("/cart/{id}/items", req.cartId())
                .bodyValue(new CartItem(tuple.getT1(), tuple.getT2(), req.quantity()))
                .retrieve().bodyToMono(AddItemResult.class));
    }
}
```

The Companion: Service Registry Pattern

The aggregator above hard-codes `http://product-service` as a hostname, which breaks the moment a service is rescaled, redeployed, or moved to a new instance. The Service Registry pattern solves this by having every service instance register itself (host, port, health status) on startup, and having callers look up healthy instances by service name at call time instead of relying on a fixed address — this is exactly the registry consulted by the Aggregator in Fig. 12 above before it fans out to Catalog, Shopping Cart, Discount, and Ordering.

pom.xml — Eureka client (registry) + Eureka server (registry itself)

```
<!-- on the registry server module -->
<dependency>
  <groupId>org.springframework.cloud</groupId>
  <artifactId>spring-cloud-starter-netflix-eureka-server</artifactId>
</dependency>

<!-- on every microservice that should be discoverable -->
<dependency>
  <groupId>org.springframework.cloud</groupId>
  <artifactId>spring-cloud-starter-netflix-eureka-client</artifactId>
</dependency>
```

RegistryApplication.java + application.yml (registry server)

```
@SpringBootApplication
@EnableEurekaServer
public class RegistryApplication {
    public static void main(String[] args) { SpringApplication.run(RegistryApplication.class, args); }
}
```

```
server:
  port: 8761
eureka:
  client:
    register-with-eureka: false   # the registry server doesn't register with itself
    fetch-registry: false
```

application.yml (product-service — registers itself on startup)

```
spring:
  application:
    name: product-service
eureka:
  client:
    service-url:
      defaultZone: http://localhost:8761/eureka
  instance:
    lease-renewal-interval-in-seconds: 10
```

AddItemAggregator.java — looking up instances by name instead of a hard-coded host

```
@Service
public class AddItemAggregator {
    private final DiscoveryClient discoveryClient;   // injected by Spring Cloud, backed by Eureka
    private final WebClient.Builder webClientBuilder;

    private WebClient clientFor(String serviceName) {
        List<ServiceInstance> instances = discoveryClient.getInstances(serviceName);
        if (instances.isEmpty())
            throw new ServiceUnavailableException(serviceName + " has no healthy instances");
        // simple round-robin / random pick across all currently-healthy instances
        ServiceInstance chosen = instances.get(ThreadLocalRandom.current().nextInt(instances.size()));
        return webClientBuilder.baseUrl(chosen.getUri().toString()).build();
    }

    public Mono<ProductDto> fetchProduct(String productId) {
        return clientFor("product-service").get().uri("/products/{id}", productId)
            .retrieve().bodyToMono(ProductDto.class);
    }
}
```

Real-Time Usage in Production

- **E-commerce checkout pages:** aggregators combine product details, live pricing, and inventory availability in one parallel fan-out call instead of a slow sequential chain, directly cutting page load latency.
- **Travel search results:** a search-results aggregator queries flight, hotel, and car inventory services in parallel and merges results, rather than querying them one after another.
- **Dashboard/BFF backends:** any "single screen, many data sources" API (this is the same underlying technique used inside BFF implementations in Pattern 5) benefits from aggregation over chaining.

- **Auto-scaling cloud deployments:** a service registry (Eureka, Consul, or Kubernetes' built-in service discovery) lets callers keep working correctly as instances are added/removed by an autoscaler, without any caller-side configuration change.

When to use: Whenever you catch yourself writing "Service A calls Service B which calls Service C which calls Service D" — stop, and introduce an aggregator that calls B, C, and D directly and in parallel instead. Pair it with a service registry (or, on Kubernetes, rely on its built-in DNS-based service discovery instead of standing up Eureka) as soon as you have more than one instance per service.

Bonus B — Message Broker, Publish-Subscribe & Async Communication

Event-Driven

The Problem

Synchronous REST calls between services mean the caller blocks until the callee responds, and both services must be up simultaneously. For workflows where the caller doesn't need an immediate answer (send a confirmation email, update a search index, notify a downstream analytics pipeline), a message broker decouples producers from consumers entirely — the producer publishes once, and any number of subscribers react independently, at their own pace, even if some are temporarily offline.

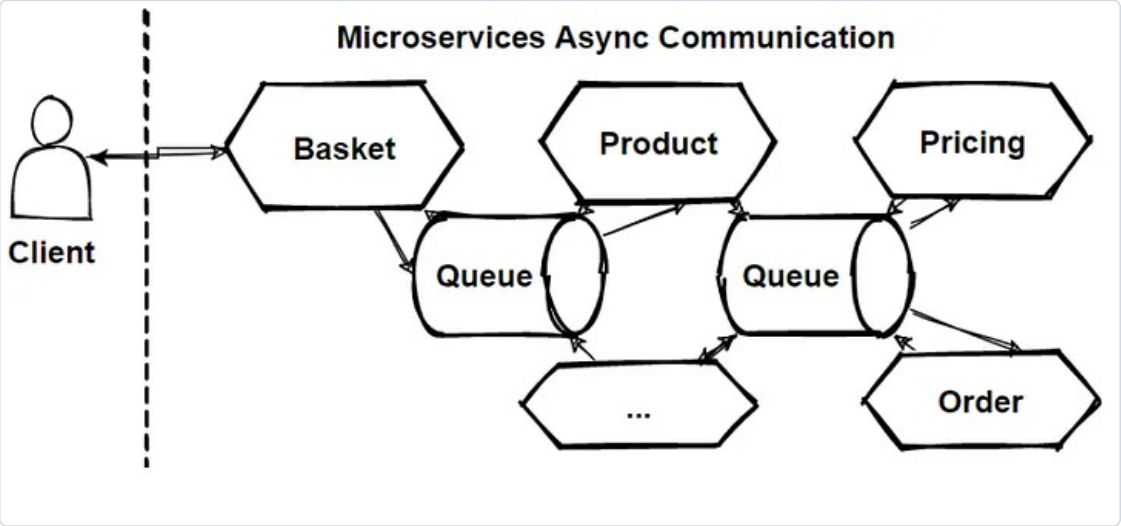


Fig. 13 — Basket, Product, Pricing, and Order services communicate exclusively through queues rather than direct calls — each service reads from and writes to a queue, with no service ever calling another service's endpoint directly.

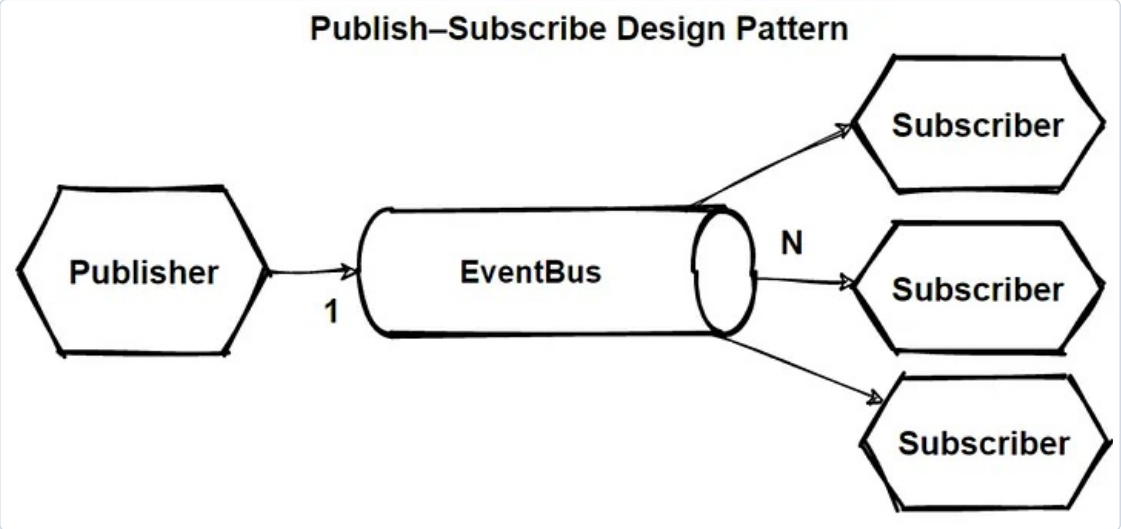


Fig. 14 — The core Publish-Subscribe primitive: one Publisher sends to an EventBus, which fans the message out to N Subscribers — none of whom the publisher needs to know about individually.

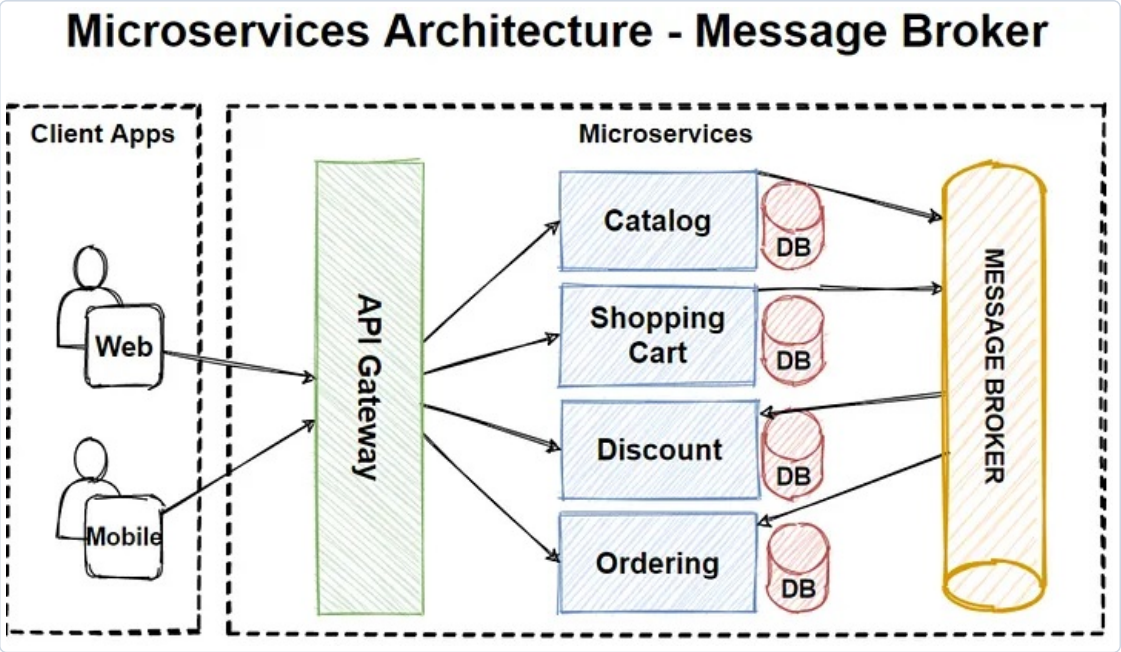


Fig. 15 — A Message Broker sits alongside the API Gateway and every microservice's own database, letting Catalog, Shopping Cart, Discount, and Ordering publish and subscribe to each other's events without direct service-to-service coupling.

pom.xml

```
<dependency>
  <groupId>org.springframework.kafka</groupId>
  <artifactId>spring-kafka</artifactId>
</dependency>
```

ProductEventPublisher.java — one publish, many independent subscribers

```
@Service
public class ProductEventPublisher {
    private final KafkaTemplate<String, ProductUpdatedEvent> kafkaTemplate;

    public void publishPriceChange(String productId, BigDecimal newPrice) {
        kafkaTemplate.send("product-updated", productId,
            new ProductUpdatedEvent(productId, newPrice, Instant.now()));
        // the publisher has zero knowledge of who (if anyone) is listening
    }
}
```

Three independent subscribers to the same topic

```
@Component
public class SearchIndexUpdater {
    @KafkaListener(topics = "product-updated", groupId = "search-indexer")
    public void onPriceChanged(ProductUpdatedEvent e) { searchIndex.reindexPrice(e.productId(), e.newPrice()); }
}

@Component
public class CartPriceRefresher {
    @KafkaListener(topics = "product-updated", groupId = "cart-service")
    public void onPriceChanged(ProductUpdatedEvent e) { cartRepository.refreshCachedPrice(e.productId(), e.newPrice()); }
}

@Component
public class PriceDropNotifier {
    @KafkaListener(topics = "product-updated", groupId = "notification-service")
    public void onPriceChanged(ProductUpdatedEvent e) { wishlistService.notifyUsersWatching(e.productId(), e.newPrice()); }
}
// each consumer group receives every message independently -- adding a 4th subscriber
// requires zero changes to the publisher or to any existing subscriber
```

Choosing a Broker: Kafka vs. RabbitMQ

Consideration	Apache Kafka	RabbitMQ
Model	Durable, replayable log partitioned per topic	Traditional message queue with exchanges/bindings and routing rules
Best fit	High-throughput event streaming, CQRS/Event Sourcing (Patterns 2 & 3), where consumers may need to replay history	Complex routing topologies, task queues, RPC-style messaging, lower operational overhead for smaller message volumes
Message		Messages are typically consumed and removed; not designed for

retention	Configurable, events can be kept indefinitely and replayed	replay
Ordering guarantee	Per-partition ordering (key by aggregate id, as used throughout this guide's Saga and CQRS sections)	Per-queue ordering, no native partitioning concept

This guide standardizes on Kafka throughout because Event Sourcing (Pattern 3) and CQRS (Pattern 2) specifically depend on the ability to replay a durable log — a capability RabbitMQ isn't designed around. RabbitMQ remains a perfectly reasonable choice for simpler pub-sub or task-queue needs where replay isn't a requirement.

Real-Time Usage in Production

- **E-commerce price-drop notifications:** a single "product updated" event fans out to search re-indexing, cart price refresh, and wishlist notification consumers — all independently, all without the product service knowing they exist.
- **Order confirmation workflows:** "order placed" publishes once; email service, SMS service, warehouse-picking service, and analytics pipeline all subscribe and react on their own schedule.
- **IoT / telemetry platforms:** device event streams publish to a broker consumed by real-time dashboards, alerting rules, and long-term storage/data-lake ingestion simultaneously (see Fig. 16 below for the general "any source, any sink" event hub shape used at platform scale).

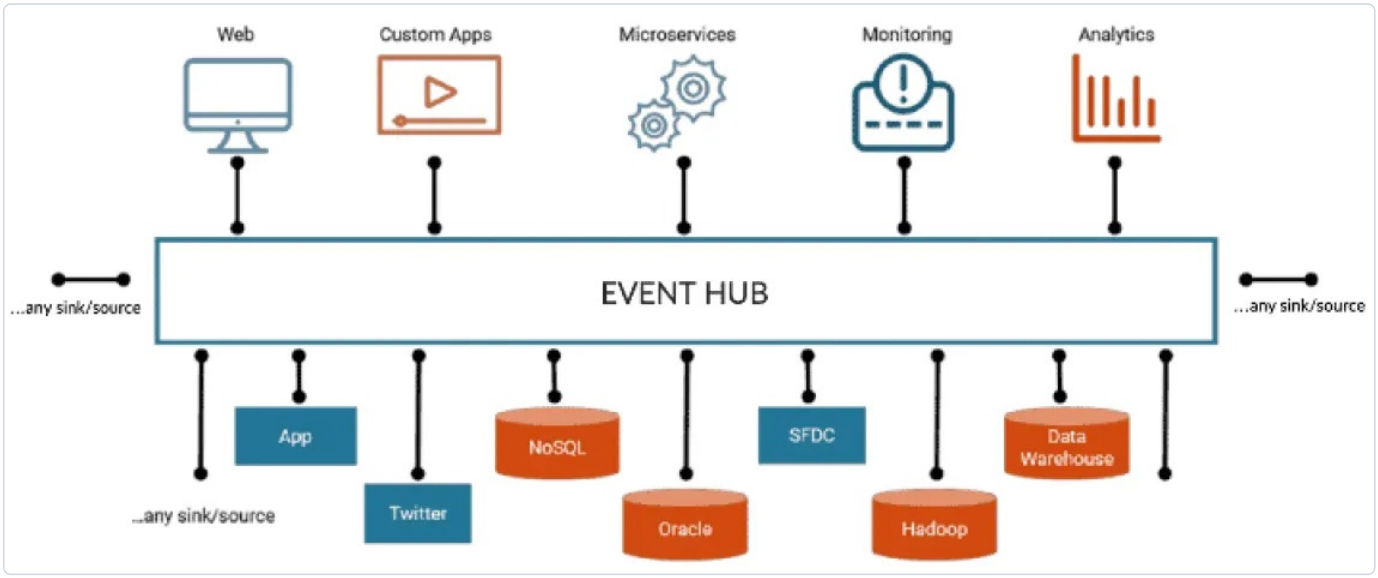


Fig. 16 — At platform scale, an Event Hub becomes the universal ingress/egress point: Web, Custom Apps, Microservices, Monitoring, and Analytics all publish and subscribe through one hub, which fans out to NoSQL, Oracle, Hadoop, SFDC, and Data Warehouse sinks.

Gotchas:

- Async communication trades immediate consistency for decoupling — the same eventual-consistency caveats from CQRS (Pattern 2) apply to every consumer of a published event.
- A "fire and forget" publisher still needs delivery guarantees — use Kafka's `acks=all` plus idempotent producer settings so a broker restart doesn't silently drop events.
- Unbounded subscriber growth on a shared topic is a feature, not a bug — but it also means you must version event schemas carefully (see Event Sourcing's schema-evolution note in Pattern 3), since you may not know every current consumer of a topic.

Bonus C — Event-Driven Microservices Architecture Platform-Wide

The Big Picture

Pulling every pattern in this guide together, a mature event-driven microservices platform looks like the diagram below: client apps (Web, Mobile, IoT) talk to a set of independently deployed microservices, each still owning its own database, but all of them also publish onto and consume from a shared event backbone — fed by both stream processing and serverless (FaaS) functions — which ultimately lands everything in a Data Lake for downstream analytics.

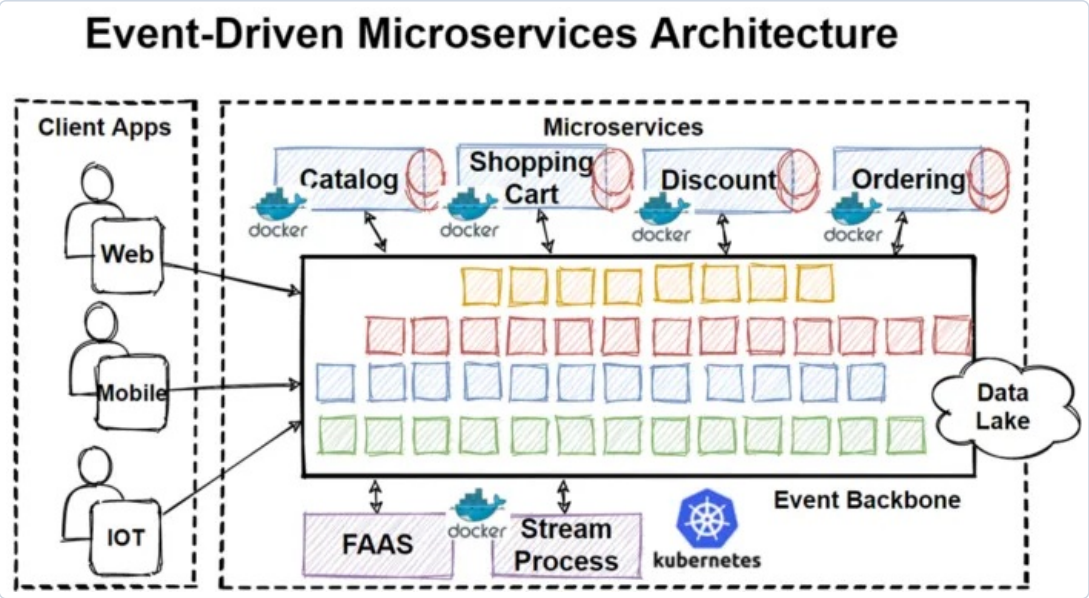


Fig. 17 — Web, Mobile, and IoT clients call Catalog, Shopping Cart, Discount, and Ordering microservices directly; all services also publish onto and consume from a shared Event Backbone (colored blocks), fed by Stream Processing and FAAS functions, ultimately landing in a Data Lake.

Real-Time Usage in Production

- **Retail/e-commerce platforms at scale:** every microservice both serves synchronous requests and emits domain events onto a shared backbone, letting new consumers (fraud detection, personalization, BI dashboards) be added later without touching existing services.
- **IoT platforms:** device telemetry flows through the same event backbone as user-facing microservice traffic, letting stream-processing jobs and serverless functions react to sensor data in near real time alongside normal application events.
- **Enterprise data platforms:** the same event backbone that powers inter-service communication doubles as the ingestion pipeline for the company's data lake, avoiding a separate ETL pipeline built on top of database change-logs.

When to use: Once a platform has enough services and enough downstream consumers (analytics, ML, monitoring) that point-to-point integrations between every pair of services becomes unmanageable — the event backbone becomes the single place new consumers plug into, rather than a new direct integration per consumer.

Bonus D — The Outbox Pattern `Reliable Event Publishing`

The Problem: Updating a Database and Publishing an Event Isn't Atomic

Every event-driven pattern in this guide — CQRS, Event Sourcing, Saga, the Event Hub — depends on one unglamorous guarantee: when a service changes its own data, the corresponding event reliably gets published. But a database write and a message-broker publish are two entirely separate systems. A naive "save to the database, then call the broker" sequence has an unavoidable gap: if the process crashes, times out, or loses network connectivity between those two steps, you end up with a change saved to the database that nobody downstream ever hears about — or, just as bad, an event published for a database write that then fails to commit. Either way, the read models, sagas, and consumers in this guide's other patterns silently drift out of sync with the truth.

The Fix: Write the Event in the Same Transaction as the Data

The Outbox Pattern solves this by never publishing directly from the request-handling path at all. Instead, the service writes the domain event into an **Outbox table** in its own database, as part of the exact same local transaction that updates the actual business entity. Since both writes happen in one transaction, they succeed or fail together — there is no gap where one could happen without the other. A separate, independent process then reads pending rows from the Outbox table and publishes them to the real event bus, on its own schedule.

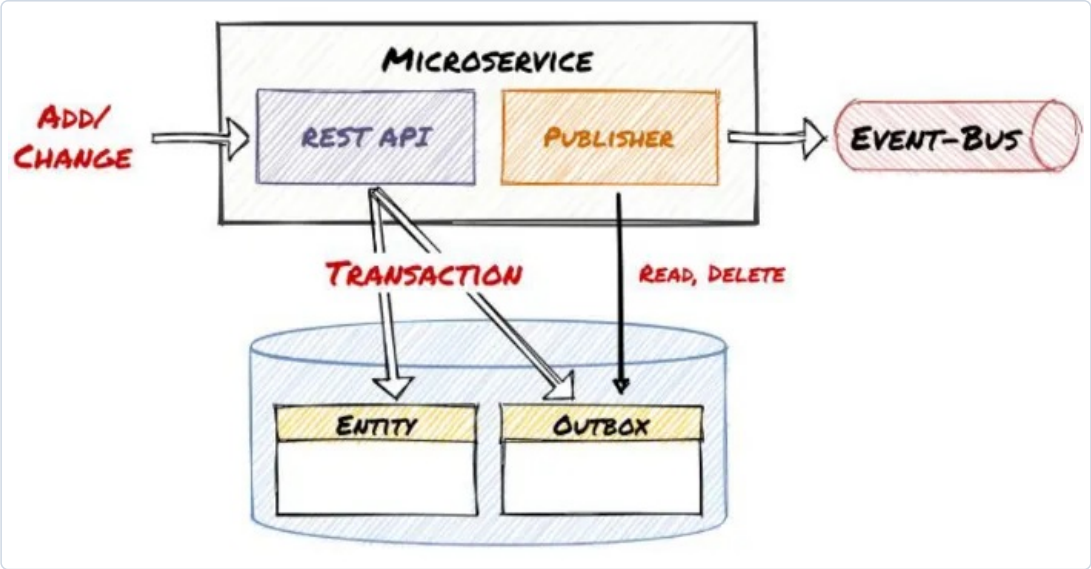


Fig. 18 — An Add/Change request hits the REST API, which writes to the Entity table and the Outbox table in one Transaction. A separate Publisher reads (and later deletes) rows from the Outbox table and forwards them to the Event-Bus — the REST API path never talks to the Event-Bus directly.

Hands-On: A Generic Outbox Implementation

V1__outbox.sql — works identically on PostgreSQL, MySQL, or SQL Server

```
CREATE TABLE outbox (
    id           UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    aggregate_id VARCHAR(64)  NOT NULL,
    event_type   VARCHAR(100) NOT NULL,
    payload      TEXT         NOT NULL,
    created_at   TIMESTAMP    NOT NULL DEFAULT now()
);
```

ProductService.java — entity write + outbox write, one local transaction

```
@Service
public class ProductService {
    private final ProductRepository productRepository;
    private final OutboxRepository outboxRepository;
    private final ObjectMapper mapper;

    @Transactional
    public Product createProduct(CreateProductRequest req) {
        Product saved = productRepository.save(Product.from(req));

        // same transaction as the line above -- either both commit, or neither does
        ProductCreatedEvent event = new ProductCreatedEvent(saved.getId(), saved.getName(), Instant.now());
        outboxRepository.save(new OutboxRow(saved.getId(), "ProductCreatedEvent", toJson(event)));

        return saved; // REST API never touches the event bus directly -- see Fig. 18
    }

    private String toJson(Object event) {
        try { return mapper.writeValueAsString(event); }
        catch (Exception e) { throw new RuntimeException(e); }
    }
}
```

OutboxPublisherJob.java — the independent process that drains the outbox on an interval

```
@Component
public class OutboxPublisherJob {
    private final OutboxRepository outboxRepository;
    private final KafkaTemplate<String, String> kafkaTemplate;

    @Scheduled(fixedDelay = 500) // predefined interval -- a CDC-based reader removes even this poll
    @Transactional
    public void publishPendingEvents() {
        List<OutboxRow> pending = outboxRepository.findTop100ByOrderByCreatedAtAsc();

        for (OutboxRow row : pending) {
            try {
                kafkaTemplate.send("product-events", row.getAggregateId(), row.getPayload()).get();
                outboxRepository.delete(row); // "Read, Delete" from Fig. 18, only after a confirmed publish
            } catch (Exception e) {
                // leave the row in place -- it will be retried on the next scheduled run
                log.warn("Failed to publish outbox row {}, will retry", row.getId(), e);
            }
        }
    }
```

```
}
```

Real-Time Usage in Production

- **Order / payment processing:** exactly the scenario the pattern is built for — an order total must be saved and its corresponding event must be published with no possibility of one happening without the other, since financial calculations and downstream billing/shipping all depend on that event arriving.
- **Inventory management:** a stock adjustment and its corresponding `StockAdjustedEvent` must never drift apart, or downstream reordering/alerting systems will act on stale stock levels.
- **Any CQRS or Event Sourcing write path in this guide** (Part I Pattern 2 & 3, Part II Sections 3–5) — the outbox pattern is the concrete mechanism that makes the "write event, then sync read model" step actually reliable, rather than merely usually-reliable.

Gotchas:

- Polling on a fixed interval (as above) adds up to that interval's worth of latency before an event is published; a Change-Data-Capture-based outbox reader (Debezium tailing the outbox table's write-ahead log) removes the poll entirely and is the standard production upgrade, as used in Part II Section 5.
- Deleting a row only after a *confirmed* publish (not before sending) is what makes the publisher itself safe to crash and restart — a row still present after a crash simply gets retried, which is why the Kafka producer above should also be configured idempotent to make that retry safe.
- The outbox table needs its own housekeeping — a long-running publisher outage lets it grow unbounded, and monitoring its row count (or oldest row's age) is a simple, effective health signal for the whole event-publishing pipeline.

When to use: Whenever a database update and an event publish must be atomic — that is, whenever losing an event or publishing a "phantom" event for a change that never actually committed would be a real business problem (financial transactions being the clearest case). For lower-stakes events where an occasional missed message is tolerable, a simpler "save then publish" without an outbox may be an acceptable, lower-complexity trade-off.

13. Pattern Selection Cheat Sheet

Symptom / Requirement	Pattern
Services stepping on each other's schema changes	Database per Microservice
Read and write workloads have very different scaling/shape needs	CQRS
Need full audit history / time-travel / rebuildable read models	Event Sourcing
A business transaction spans 3+ services with no single database	Saga (Orchestration for complex flows)
Mobile and web clients need very different aggregated payloads	Backend for Frontend
Clients need one stable entry point; need central auth/rate-limiting	API Gateway
Migrating a legacy monolith incrementally, with rollback safety	Strangler Fig
A downstream dependency's failures are cascading upstream	Circuit Breaker
Config changes require redeploys; need per-environment overrides	Externalized Configuration
A provider's API change silently broke a consumer in production	Consumer-Driven Contract Testing

Patterns compose. A realistic production checkout flow in this guide's e-commerce example actually uses all ten together: API Gateway routes the request → BFF aggregates the response shape → Saga orchestrates Order/Customer/Product → each writes to its own database (Pattern 1) → Order's write side publishes to Kafka, projected into a Cassandra read model (CQRS) → Order's full history is Event Sourced for audit → every inter-service call is wrapped in a Circuit Breaker → all services pull config from Config Server → and Pact contracts protect the User↔Order integration boundary in CI.

14. Common Pitfalls Across All Patterns

1. **Distributed monolith trap:** splitting services by technical layer instead of business capability, so a single feature change still requires deploying five services in lockstep. Split along domain boundaries (DDD bounded contexts), not "controllers/services/repositories."
2. **No idempotency strategy:** Kafka, HTTP retries, and Saga compensations all imply at-least-once delivery somewhere. Every command handler and event consumer needs a dedup/idempotency key strategy from day one, not bolted on after the first duplicate-charge incident.
3. **Skipping distributed tracing:** once you have 5+ services, request logs scattered across services are useless without correlation IDs. Wire up OpenTelemetry / Micrometer Tracing with a `traceId` propagated through both HTTP headers and Kafka message headers from the start.
4. **Treating eventual consistency as a bug:** CQRS, Saga, and Event Sourcing all introduce a window where reads lag writes. Design the UI/UX and the tests around this explicitly, don't discover it in production.
5. **Under-provisioning local dev environments:** the full stack in Section 2 (Kafka + 3 databases + Cassandra) is heavy. Use the "lite profile" and Testcontainers for day-to-day development; reserve the full Docker Compose stack for integration test suites and pre-release verification.
6. **No contract or schema versioning strategy:** Kafka event schemas and REST API DTOs will change. Adopt Avro/Protobuf with a Schema Registry for Kafka payloads, or at minimum a strict additive-fields-only policy plus Pact contracts (Pattern 10) for REST boundaries.

— End of Guide — Extended Hands-On Edition, July 2026 —